Agentic_AI: 3

1. What is an AI Agent?

An **AI agent** is a software system that uses an AI model (usually a **Large Language Model, LLM**) to **understand a goal**, **decide what steps to take**, **use tools** to act in the real world (search, run code, call APIs, read files) and **repeat until the goal is complete**, with little or no human help at every step.

A normal AI chat model only **answers**. An AI agent **acts**. You give it a goal like "Book the cheapest flight to Delhi next Friday and add it to my calendar", and the agent plans the steps, searches flights, compares prices, books, and updates the calendar.

Simple definition: AI Agent = LLM (brain) + Tools (hands) + Memory + Planning loop.

Real-life analogy

Think of a **personal assistant** in an office. You say, "Arrange a meeting with the client." The assistant checks calendars, picks a time, sends emails, books a room and tells you when everything is done. You did not give step-by-step instructions. An AI agent works the same way.

Short history

- **Classical AI agents**: rule-based and search-based programs (thermostats, game bots, robots).
- **Reinforcement learning agents**: learn by trial and error (AlphaGo, game-playing agents).
- **LLM-based agents**: since 2023, language models are used as the reasoning engine with tools (AutoGPT, LangChain agents, ChatGPT plugins, coding agents).
- **Today**: agents with tool calling, long context, computer use, coding ability, MCP and multi-agent teams are used in production.

2. Chatbot vs LLM vs Workflow vs Agent

Term	Meaning	Example
LLM	A trained model that predicts text. Has no tools or memory by itself.	A raw model API
Chatbot	An interface around an LLM or rules for conversation. Mostly answers questions.	FAQ bot, basic website chat
AI Workflow	LLM calls and tools connected in a **fixed path written by a developer** (step 1, then 2, then 3).	Summarize email, classify it, then send to a team
AI Agent	The LLM **decides dynamically** which steps and tools to use, and loops until the goal is met.	Coding agent that reads a repo, edits files, runs tests, fixes errors

Key difference: In a **workflow**, the code controls the path. In an **agent**, the model controls the path. Workflows are more predictable and cheaper. Agents are more flexible but harder to control. Use the simplest solution that works.

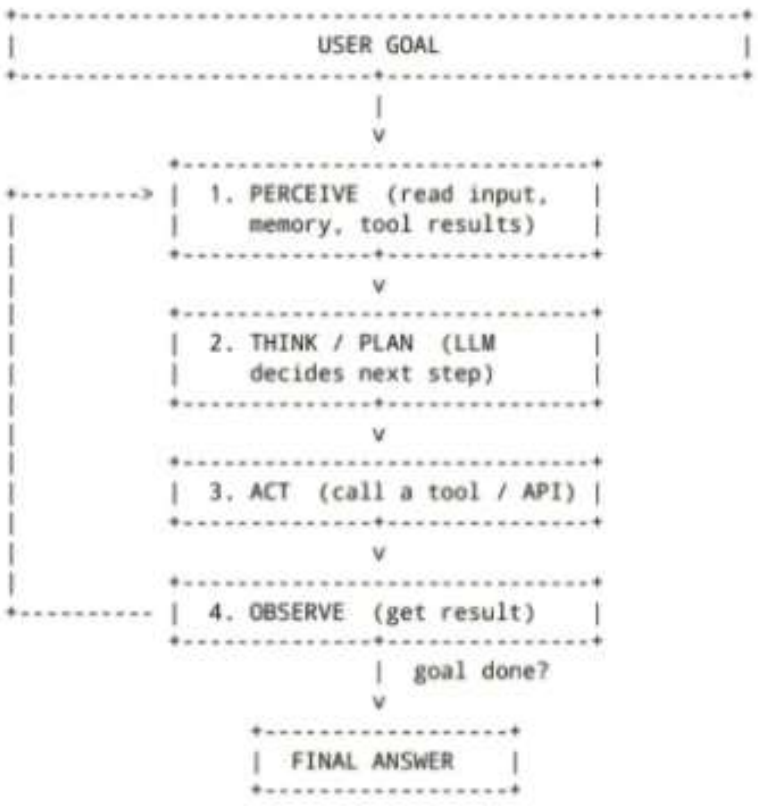
Feature	Chatbot	Agent
Goal	Reply to the message	Complete a task
Steps	Usually one reply	Many steps with a loop
Tools	None or few	Search, code, APIs, files, browser
Memory	Chat history only	Short-term and long-term memory
Autonomy	Low	Medium to high
Risk	Wrong answer	Wrong answer plus wrong **action**

3. Key Characteristics of Agents

Characteristic	Explanation
Autonomy	Works on its own to reach a goal without a human guiding each step.
Goal-oriented	Has an objective and measures progress towards it.
Perception	Takes input from the environment: user text, files, web pages, API results, screenshots, sensors.
Reasoning	Thinks about what to do next, breaks big problems into smaller steps.
Action / Tool use	Does things: calls functions, searches, writes code, sends emails, clicks buttons.
Memory	Remembers past steps, user preferences and earlier results.
Adaptability	Changes the plan when something fails or new information arrives.
Reactivity and proactivity	Responds to changes in the environment and also takes initiative.

4. How an Agent Works (The Agent Loop)

Every agent runs a loop, often called the **perceive - think - act - observe** loop (similar to the **ReAct** pattern: Reason + Act).



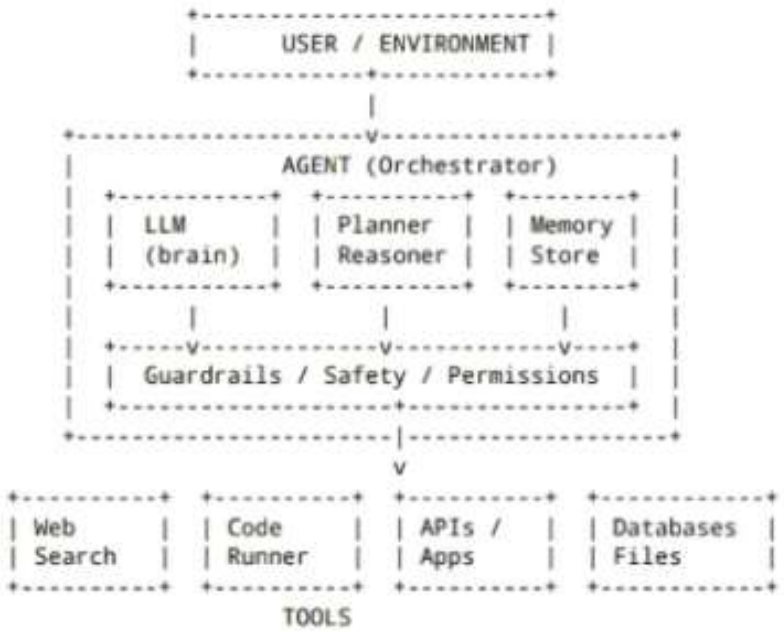
Step-by-step example

Goal: "Find the top 3 AI news today and email me a summary."

Step	Thought	Action	Observation
1	I need current news.	search_web("AI news today")	10 article links returned
2	Open the top articles.	fetch_page(url1..url3)	Page text received
3	Summarize each in 2 lines.	(LLM writes summary)	Summary ready
4	Send the email.	send_email(to, subject, body)	Email sent successfully
5	Goal complete.	Return final answer	Done

The loop stops when the model decides the goal is done, when a **maximum step limit** is reached, or when a human stops it. Always set a step limit to avoid infinite loops and high cost.

5. Components of an AI Agent



Component	Role
LLM (Brain)	Understands instructions, reasons, chooses tools, writes the final answer. Quality of the model strongly affects agent quality.
System prompt / Instructions	Defines the agent's role, rules, style, what it may and may not do.
Tools	Functions the agent can call: search, calculator, code execution, database query, email, calendar, browser.
Memory	Short-term (current conversation) and long-term (stored knowledge, preferences, past tasks).
Planner	Breaks the goal into sub-tasks and orders them. Can be the LLM itself.
Knowledge base (RAG)	Private documents the agent can search to give grounded answers.
Orchestrator / Runtime	The code that runs the loop, sends tool results back to the model, handles errors, retries and limits.
Guardrails	Checks that block unsafe inputs, unsafe actions and wrong outputs. Includes human approval for risky steps.
Observability	Logs and traces of every step for debugging and evaluation.

6. Types of AI Agents

A) Classical types (from AI textbooks)

Type	How it decides	Example
Simple reflex agent	Uses only the current input with if-then rules. No memory.	Thermostat, automatic door
Model-based reflex agent	Keeps an internal model of the world to handle things it cannot see directly.	Robot vacuum remembering room layout
Goal-based agent	Chooses actions that move towards a goal; uses search and planning.	GPS navigation
Utility-based agent	Chooses the action with the best **utility** (best trade-off of speed, cost, safety).	Ride app choosing best route and price
Learning agent	Improves from experience using feedback.	Recommendation systems, game AI

B) Modern types (by design)

Type	Description
Single-agent	One agent with tools handles the whole task.
Multi-agent	Several specialized agents (researcher, writer, reviewer) collaborate.
Conversational agent	Talks with users, answers and takes light actions (support assistant).
Task / Autonomous agent	Given a goal, works in the background for a long time (research agent, coding agent).
Human-in-the-loop agent	Pauses for human approval before important actions.
Computer-use / Browser agent	Looks at the screen and controls mouse, keyboard and browser like a person.
Embodied agent	Controls a physical body such as a robot.

C) By level of autonomy

- **Level 1: Assistive**: suggests, human does the action.
- **Level 2: Semi-autonomous**: acts but asks for approval on key steps.
- **Level 3: Autonomous**: completes the task end to end and reports back.
- More autonomy means more value but also more risk. Start low and increase gradually.

7. Tools and Function Calling

Tools are what turn a language model into an agent. A tool is simply a function the model is allowed to request, for example search_web(query) or get_weather(city). The model never runs the tool itself. It only **asks** for it; your code runs it and returns the result.

How function (tool) calling works

- You describe each tool: **name**, **description** and **input schema** (JSON Schema).
- You send the user message and the tool list to the model.
- The model decides if a tool is needed and replies with a **tool call** containing the tool name and arguments.
- Your code runs the tool and sends the **tool result** back to the model.
- The model reads the result and either calls another tool or writes the final answer.

```
User --> [ Your App ] --> [ LLM ]
                            |  "call get_weather(city='Pune')"
                            v
          [ Your App runs get_weather('Pune') ]  --> real weather API
                            |  result: {temp: 29, sky: clear}
                            v
                        [ LLM ]  --> "It is 29 C and clear. No umbrella needed."
```

A tool definition (JSON Schema)

```
{
  "name": "get_weather",
  "description": "Get the current weather for a city. Use when the user asks
                  about weather or temperature.",
  "input_schema": {
    "type": "object",
    "properties": {
      "city": { "type": "string", "description": "City name, e.g. Pune" }
    },
    "required": ["city"]
  }
}
```

Common tool categories

Category	Examples
Information	Web search, Wikipedia, news API, document search (RAG), database queries
Computation	Calculator, Python code interpreter, data analysis
Communication	Send email, Slack message, SMS, calendar invite
Files and apps	Read/write files, Google Drive, Notion, GitHub, Jira
Computer control	Browser automation, screenshot, mouse and keyboard
Other agents	Calling another agent as a tool

Writing good tools

- Give the tool a **clear name** and a **detailed description** that explains when to use it and when not to.
- Keep the number of tools small and focused. Too many similar tools confuse the model.
- Return **short, useful output**. Huge outputs waste the context window.
- Return **clear error messages** so the agent can fix its mistake and retry.
- Validate all arguments in your code. Never trust the model blindly.

8. Memory in Agents

A language model has no memory by itself. Every call starts fresh, and it only sees what you put in the **context window**. Agent memory is the system we build around the model to remember things.

Short-term memory

The **current conversation and recent tool results** stored in the context window. It is fast and complete, but limited by size and disappears after the session ends. When it gets full, we **summarize** older messages or drop less useful ones.

Long-term memory

Information stored **outside the model** and retrieved when needed, so the agent remembers across sessions. Common storage: a **vector database** (semantic search), a normal database, or plain files.

Memory type	What it stores	Example
Working memory	What the agent is doing right now	Current plan and last tool outputs
Episodic memory	Past events and experiences	"Last week the user asked for a sales report in Excel"
Semantic memory	Facts and knowledge	User's name, company, preferences
Procedural memory	How to do tasks (skills, instructions)	Saved workflow for deploying an app

Vector memory (how it works)

- Text is converted into a list of numbers called an **embedding** that captures its meaning.
- Embeddings are stored in a **vector database** (Pinecone, Chroma, FAISS, pgvector, Weaviate, Qdrant).
- For a new query, we embed it and find the **most similar** stored items using cosine similarity.
- The retrieved items are added to the prompt so the model can use them.

Tip: Do not store everything. Store **useful, verified facts**, add timestamps and allow users to view and delete their memories for privacy.

9. Planning and Reasoning Patterns

Pattern	Explanation
Chain of Thought (CoT)	The model reasons step by step before answering. Improves accuracy on logic and math.
ReAct (Reason + Act)	The agent alternates: Thought, Action, Observation, Thought ... This is the most common agent loop.
Plan-and-Execute	First create a full plan, then execute each step (possibly with a cheaper model). Re-plan if something changes.
Reflection / Self-critique	The agent reviews its own output, finds errors and improves it.
Tree of Thoughts	Explore multiple reasoning paths, evaluate them and keep the best.
Task decomposition	Break a big goal into smaller sub-tasks that are easier to solve.

ReAct example

```
Question: Who is older, the CEO of Company A or the CEO of Company B?

Thought 1: I need to find the CEO of Company A and their age.
Action 1 : search("CEO of Company A")
Observation 1: The CEO is John Smith, born 1970.

Thought 2: Now find the CEO of Company B.
Action 2 : search("CEO of Company B")
Observation 2: The CEO is Maria Lopez, born 1965.

Thought 3: 1965 is earlier than 1970, so Maria is older.
Final Answer: The CEO of Company B (Maria Lopez) is older.
```

Plan-and-Execute example

```
Goal: Write a market report on electric scooters.

PLAN
 1. Search for market size and top companies
 2. Collect pricing and customer reviews
 3. Analyze trends
 4. Write the report with headings
 5. Review and fix mistakes

EXECUTE  ->  run step 1, 2, 3 ... using tools, update the plan if needed.
```

SUBTOPIC: Introduction to Agentic AI

1) What is Agentic AI ?

Agentic AI refers to AI systems that can autonomously plan, make decisions, use tools, take actions and learn from the environment to achieve specific goals with minimal human intervention.

> "It's not just about generating answers, but about taking actions towards achieving a goal."

2) Key Characteristics

- Autonomy – can operate independently
- Goal-oriented – works towards specific goals
- Reasoning – can analyze, plan and decide
- Tool Use – can use external tools/APIs
- Memory – can remember past interactions
- Adaptability – can learn and improve
- Action Execution – takes real-world actions
- Continuous Interaction – interacts with environment and users

3) Evolution of AI

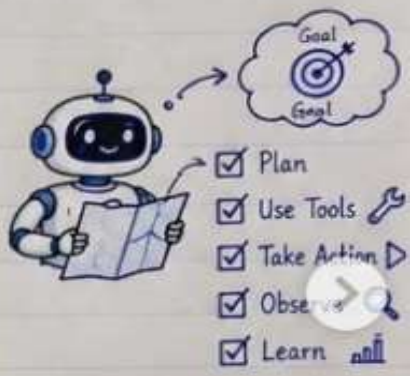
Traditional AI
- Rule-based
- Predefined logic
- Handles specific tasks
- No learning

→

Generative AI (LLMs)
- Generates text, images, code etc.
- Understands and generates content
- Mostly reactive
- Limited real-world actions

→

Agentic AI
- Plans and reasons
- Uses tools
- Takes actions
- Learns from experience
- Works towards goals autonomously



4) Comparison Table

Feature	Traditional AI	Generative AI	Agentic AI
Main Purpose	Solve specific tasks	Generate content	Achieve goals
Approach	Rule-based / ML models	Deep learning (LLMs)	Planning + Reasoning + Action + Learning
Autonomy	Low	Low (mostly reactive)	High
Tool Use	Limited	Limited	Extensive
Memory	Minimal	Limited context	Short-term + Long-term
Output	Classification / Prediction	Text, images, code etc.	Actions + Results
Real-world Interaction	No	Indirect	Yes (via tools/APIs)
Example	Spam filter	ChatGPT	AI Research Agent, Coding Agent, Automation Agent

5) Real-world Examples of Agentic AI

- AI research assistant – searches, reads papers, summarizes and writes report
- Coding agent – writes, tests and debugs code
- Task automation agent – manages emails, schedules, files etc.
- Travel planning agent – books tickets, hotels, creates itinerary
- Business agent – analyzes data and takes business actions
- Personal assistant agent – handles daily tasks using tools

(6) Why is Agentic AI Important?

- Solves complex multi-step problems
- Increases productivity and automation
- Can work across multiple tools and systems
- Enables real-world impact
- Foundation for future AI-driven applications

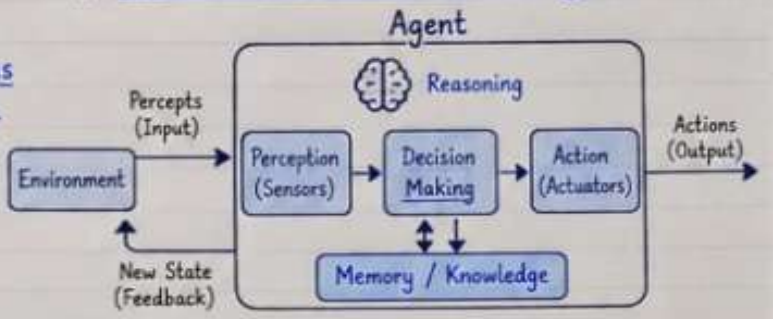
SUBTOPIC: AI Agents and Their Types

1) What is an AI Agent ?

An AI agent is an intelligent system that perceives its environment, makes decisions and takes actions to achieve specific goals.

> "An AI agent is an entity that can sense, reason, act and learn to achieve goals autonomously."

2) General Structure of an AI Agent



3) Types of AI Agents

1) Reactive Agents

- Simple agents that respond to current percepts only.
- No memory of past states.
- Work on condition-action rules.

Example: Automatic doors, basic chatbots.



2) Goal-based Agents

- Use goals to decide actions.
- Consider future states and plan actions.
- More intelligent than reactive agents.

Example: Navigation system (e.g., Google Maps) that finds a path to a destination.



3) Utility-based Agents

- Choose actions that maximize a utility function.
- Handle trade-offs between multiple goals.
- Useful when there are multiple possible outcomes.

Example: Self-driving cars (balance safety, speed, comfort).



4) Learning Agents

- Improve performance over time from experience.
- Can learn from past interactions.
- Use techniques like reinforcement learning, supervised learning etc.

Example: Personal assistants, recommendation systems.



4) Comparison of Agent Types

Feature	Reactive Agent	Goal-based Agent	Utility-based Agent	Learning Agent
Uses Past Experience	No	Partially	Partially	Yes
Uses Goals	No	Yes	Yes	Yes
Optimizes Outcomes	No	No	Yes	Yes
Can Learn	No	No	No	Yes
Complexity	Low	Medium	Medium	High
Example	Thermostat	GPS Navigation	Self-driving car	ChatGPT with memory

5) Key Takeaways

- Different types of agents are used based on the problem complexity.
- Reactive agents are simple, learning agents are most advanced.
- Agentic AI systems often combine multiple capabilities (e.g., goal-based + learning + tools).

★ Important Point:
Agentic AI is not limited to one type of agent. Real-world systems often use a combination of goal-based, utility-based and learning approaches.

SUBTOPIC: Agentic AI Architecture

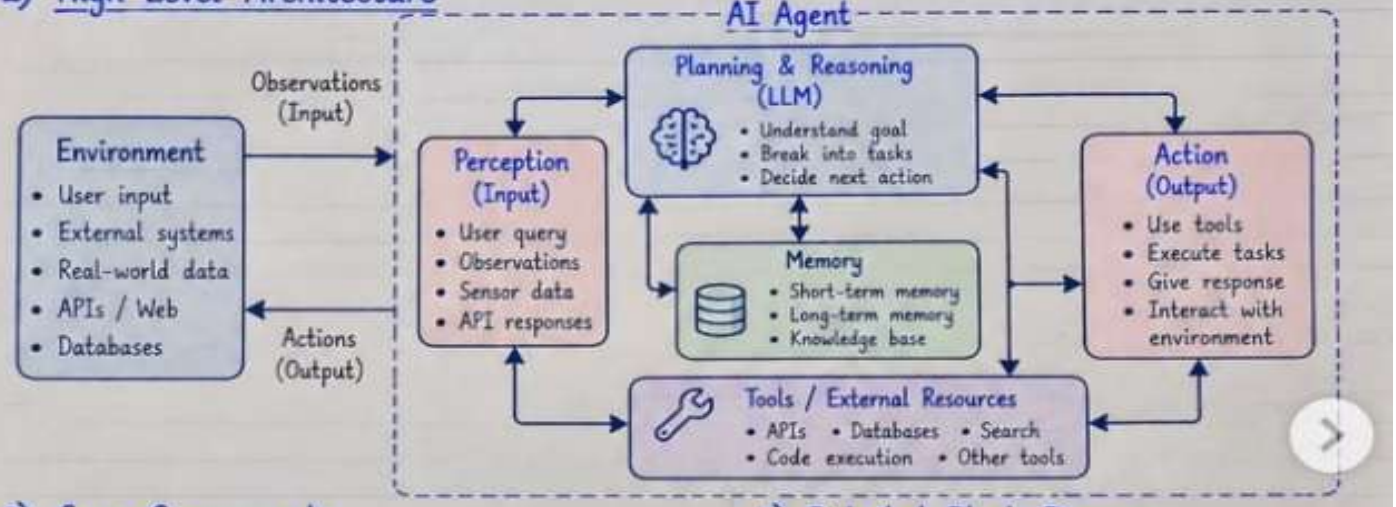
1) What is Agentic AI Architecture ?

Agentic AI architecture is the overall structure and design that enables an AI agent to perceive, reason, plan, use tools, remember past information and take actions in an environment to achieve specific goals autonomously.

Key Idea :

- Combines LLMs + Memory + Tools + Planning + Execution + Feedback loop
- Designed for autonomous goal achievement
- Can work with real-world systems and data
- More than just generating text

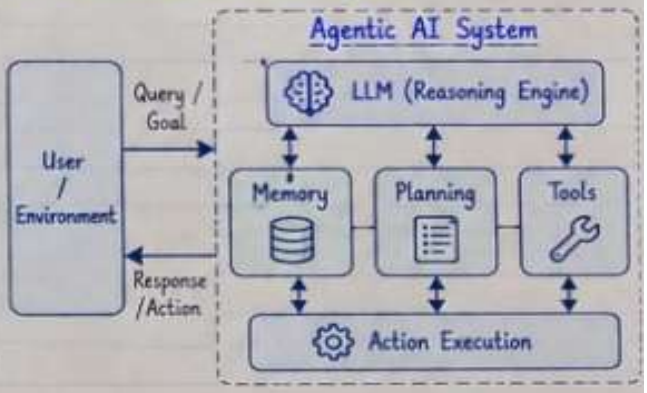
2) High Level Architecture



3) Core Components

1	Perception Module	• Takes input from user/environment • Processes and understands the input
2	Planning & Reasoning Module	• Uses LLM to analyse, plan and decide • Breaks complex goals into sub-tasks
3	Memory Module	• Stores past interactions, knowledge and context • Helps in maintaining long-term goals
4	Tools / Action Module	• Uses external tools, APIs and resources • Executes planned actions
5	Environment	• Real-world or digital environment • Provides feedback based on agent's actions

4) Detailed Block Diagram



5) Example Workflow

1. Receive Goal (User query) → 2. Understand & Plan (LLM) → 3. Retrieve Relevant Memory → 4. Use Tools (APIs / Search / Code) → 5. Take Action & Get Result → 6. Observe & Improve

6) Key Takeaways

- Agentic AI architecture integrates perception, reasoning, memory, tools and action.
- LLMs act as the core reasoning and planning engine.
- Tools and memory make agents capable of real-world task execution.
- The architecture is modular and can be extended based on use case.

★ Important Point :

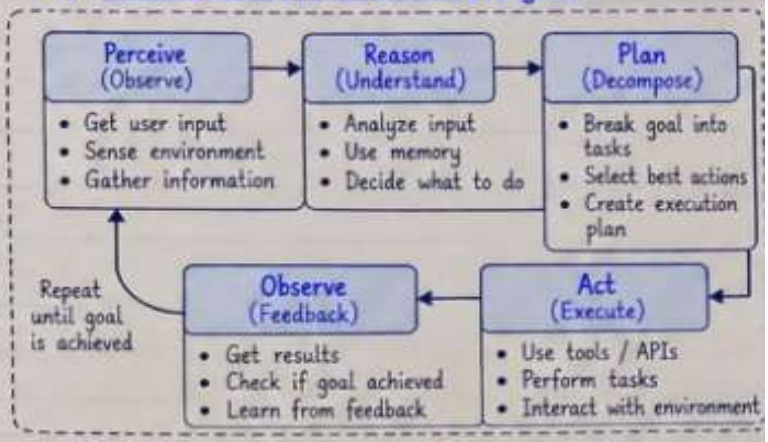
- Different frameworks (LangChain, LangGraph, CrewAI etc.) implement variations of this architecture.
- The core components remain similar across most agentic AI systems.

SUBTOPIC: Agentic AI Workflow and Reasoning

1) What is Agentic AI Workflow ?

- It is the step-by-step process followed by an AI agent to achieve a goal.
- It involves perceiving the environment, reasoning about the information, planning actions, executing those actions and observing the results.
- The workflow runs in a continuous loop until the goal is achieved.

2) Overall Workflow of an AI Agent



3) Reasoning in Agentic AI

- Reasoning is the ability of the agent to think, analyze and decide the next best action based on current information, past experience and the goal.
- LLMs are used for reasoning using techniques like Chain-of-Thought, ReAct and self-reflection.

Key Reasoning Steps

1. Understand the current state (context)
2. Identify the goal
3. Retrieve relevant knowledge from memory
4. Analyze possible actions
5. Choose the best action
6. Evaluate the outcome and adapt

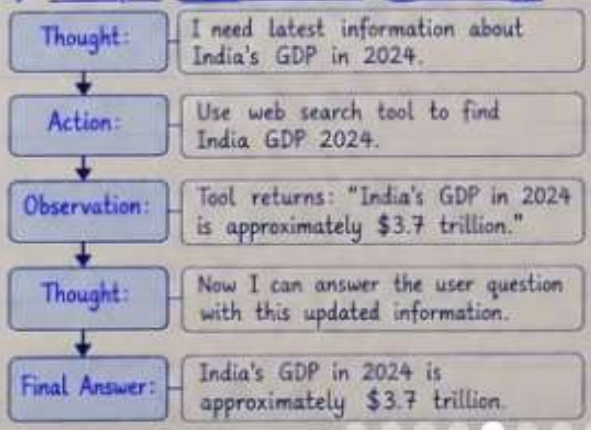
4) ReAct Pattern (Reason + Act)

- ReAct (Reasoning + Acting) is a popular approach where the agent alternates between reasoning and taking actions.
- It helps the agent use external tools and observe real-world feedback.
- Useful for tasks like search, coding, data analysis etc.

ReAct Workflow



5) Example: ReAct in Action (Search Task)



6) Other Reasoning Techniques

- Chain-of-Thought (CoT) – step by step reasoning
- Tree of Thoughts (ToT) – explore multiple reasoning paths
- Self-Consistency – generate multiple answers and choose best
- Self-Reflection – evaluate and improve own output
- Plan-and-Execute – plan the full task first, then execute
- ReAct – interleave reasoning and acting
- Reflexion – learn from past mistakes

7) Key Takeaways

- Agentic AI workflow is a continuous loop.
- Reasoning helps the agent make better decisions.
- ReAct is widely used for tool-using agents.
- Different reasoning techniques are useful for different tasks.
- The agent adapts based on feedback from the environment.

★ **Important Point:**
A good agent doesn't just generate an answer it reasons, uses tools, takes actions and learns from feedback.

SUBTOPIC: LLMs and Prompt Engineering for Agents

1) Role of LLMs in Agentic AI

- LLMs act as the brain of an agent.
- They help in understanding user queries, reasoning, planning, tool use and generating responses.
- LLMs can break complex tasks into sub-tasks, decide which tools to use and analyze results.
- They work with memory, tools and environment to achieve goals autonomously.

2) LLM vs Traditional Models

Feature	Traditional ML Models	LLMs (for Agents)
Input Type	Structured data	Natural language
Output	Fixed / classification	Open-ended text
Knowledge	Limited, trained data	Large world knowledge
Reasoning	Rule-based	Advanced reasoning
Flexibility	Task-specific	General purpose
Use in Agent	Limited	Core component

3) Types of Prompts

1. System Prompt – Sets the role, behavior and rules.
2. User Prompt – Actual query or instruction.
3. Context Prompt – Provides background information.
4. Few-shot Prompt – Gives examples to guide output.
5. Chain-of-Thought Prompt – Encourages step-by-step reasoning.
6. Tool-use Prompt – Helps LLM decide when and how to use tools.
7. Output Format Prompt – Enforces structured output (JSON etc.)

4) Anatomy of a Good Prompt

- **Role** → Define the agent's role (e.g., You are a research assistant)
- **Context** → Provide necessary background (e.g., topic, data, goal)
- **Task** → Clearly state what to do (e.g., summarize, analyze, plan)
- **Constraints** → Specify limitations or rules (e.g., use only given data)
- **Output Format** → Ask for structured response (e.g., JSON, bullet points, table)

5) Prompting Techniques for Agents

- Zero-shot Prompting – Directly ask without examples.
- Few-shot Prompting – Provide examples in the prompt.
- Chain-of-Thought (CoT) – Ask for step-by-step reasoning.
- ReAct Prompting – Combine reasoning and action.
- Self-Consistency – Generate multiple answers and pick best.
- Reflection Prompting – Ask model to review and improve its own answer.
- Tool Calling Prompting – Instruct model to use specific tools/APIs.
- Structured Prompting – Ask for output in JSON or specific format.

6) Example Prompts

Simple Prompt

```
You are a helpful research assistant.
Summarize the given article in 5 bullet points.
Article: <text>
```

Prompt for Tool Use (Function Calling)

```
You are an AI agent with access to a web search tool.
When the user asks for latest information, use the
search_tool(query) function. After getting the result,
analyze it and provide a clear answer in bullet points.
```

7) Structured Output (JSON)

Agents often need structured output to work with tools.

Example Prompt:

```
Extract the name, date and location from the following text
and return the output in JSON format.
Text: "OpenAI launched GPT-5 on 10 July 2024 in USA."
```

Expected Output:

```
{
  "name": "GPT-5",
  "date": "10 July 2024",
  "location": "USA"
}
```

8) Best Practices

- Be clear and specific in instructions.
- Provide necessary context.
- Use structured output formats.
- Add examples for better accuracy.
- Set constraints to avoid unwanted results.
- Test and iterate prompts.
- Combine with memory and tools for better agent performance.

9) Key Takeaways

- LLMs are the core reasoning engine in agentic AI.
- Prompt engineering significantly improves agent performance.
- Different prompting techniques are useful for different scenarios.
- Structured outputs (e.g., JSON) help in tool integration and automation.

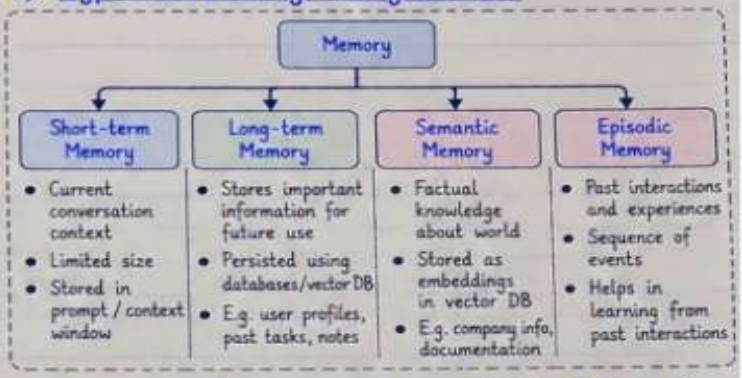
★ **Important Point:**
A well-designed prompt can make the agent more accurate, safer and capable of using tools effectively.

SUBTOPIC: Memory Systems in Agentic AI

1) Why Memory is Important ?

- LLMs are stateless – they don't remember past conversations by default.
- Memory helps an agent remember past interactions, user preferences, important facts and context.
- It improves continuity, personalization and decision making in long-term tasks.
- Without memory, the agent may repeat mistakes or give inconsistent responses.

2) Types of Memory in Agentic AI

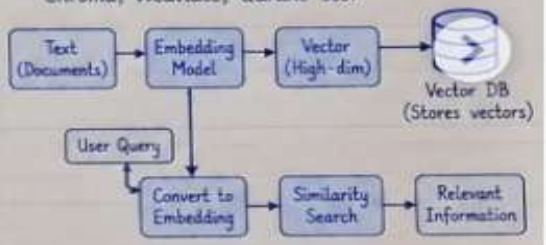


3) Short-term vs Long-term Memory

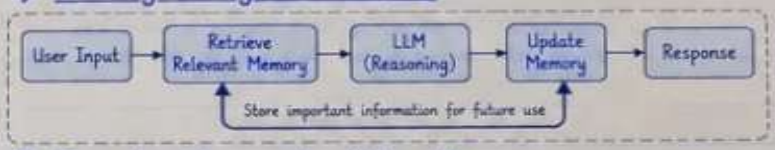
Feature	Short-term Memory	Long-term Memory
Purpose	Maintain current context	Store important info for future use
Duration	Temporary (current session)	Persistent (across sessions)
Storage	Prompt / context window	Database / Vector DB
Capacity	Limited (few thousand tokens)	Large (scalable)
Examples	Recent messages, current task	User preferences, past projects, important facts

4) Semantic Memory (Vector Database)

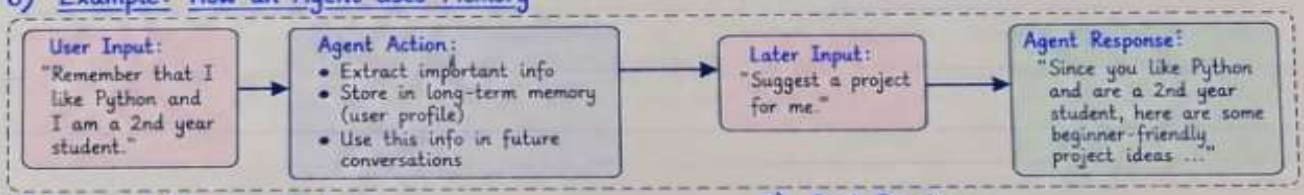
- Stores information as embeddings (vector representations) instead of plain text.
- Helps in finding relevant information using semantic search (meaning-based search).
- Common Vector Databases: Pinecone, FAISS, Chroma, Weaviate, Qdrant etc.



5) Memory in Agent Workflow



6) Example: How an Agent Uses Memory



7) Memory Storage Techniques

Technique	Description	Use Case
In-Memory	Store in prompt/context	Short-term conversation
Key-Value Store	Simple structured storage	User preferences, settings
Vector Database	Store embeddings	Semantic search, document retrieval
Relational Database	Structured data (SQL)	User profiles, logs, tasks
Graph Database	Store relationships	Knowledge graphs, complex relations
File Storage	Store text/documents	Notes, documents, history

8) Best Practices

- Store only important and relevant information.
- Use a combination of short-term + long-term memory.
- Use vector databases for unstructured data.
- Regularly update and clean old memories.
- Limit sensitive data and handle privacy carefully.
- Use summarization to compress long histories.
- Design memory retrieval to get only relevant context.

9) Key Takeaways

- Memory makes agents more useful and personalized.
- Different types of memory serve different purposes.
- Vector databases enable semantic memory.
- Good memory management improves accuracy, consistency and user experience.

SUBTOPIC: Tools, APIs and Function Calling

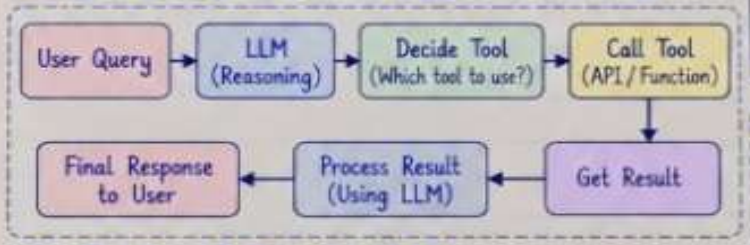
1) Why Tools are Important in Agentic AI ?

- LLMs are powerful but have limitations.
- They cannot directly access real-time data, databases or external systems.
- Tools and APIs allow agents to interact with the real world and perform useful actions.
- With tools, agents can search the web, fetch data, run code, query databases, send emails etc.
- This makes agents more capable, reliable and useful for real-world tasks.

2) Types of Tools

🌐	Web Search Tools	Search the internet for latest information. (e.g., Google Search API, SerpAPI)
🗄	Database Tools	Query structured data from databases. (e.g., MySQL, PostgreSQL)
⚙	Code Execution Tools	Run code snippets (Python, etc.). (e.g., Python REPL, Jupyter)
📄	File & Document Tools	Read, write and analyze files. (e.g., PDF reader, File system)
✉	Communication Tools	Send emails, messages, notifications. (e.g., Gmail API, Slack API)
☁	Custom Tools / APIs	Domain-specific tools for special tasks. (e.g., weather API, booking API)

3) How Tool Use Works ? (General Flow)



4) Function Calling in LLMs

- Function calling allows the LLM to decide when to call a function/tool and with what parameters.
- The LLM does not execute the function itself, it only generates a structured request (usually in JSON).
- Our application/backend receives this request, executes the function and returns the result to the LLM.
- LLM then uses the result to generate a final answer.

5) Example: Function Calling (Get Weather)

User Query: "What is the weather in Kanpur today ?"

LLM Output (Function Call Request):

```
{
  "name": "get_weather",
  "arguments": {
      "location": "Kanpur"
  }
}
```

LLM decides to call the function with required parameters.

Function Execution (Backend):

```
result = get_weather(location="Kanpur")
```

Final Response (by LLM using result):

"The current weather in Kanpur is 28°C with clear skies."

6) Common APIs Used with Agents

Purpose	API / Tool	Description
Web Search	Google Search API (SerpAPI)	Search real-time information
Weather	OpenWeather API	Get weather data
Maps	Google Maps API	Location, routes, places
Database	MySQL / PostgreSQL APIs	Query structured data
File Handling	Python file tools	Read/write documents
Email	Gmail API	Send and read emails
Custom	Any REST API	Domain specific tasks

7) Best Practices for Tool Use

- Use clear and well-defined function descriptions.
- Provide correct parameter schema (type, description, required).
- Handle errors and invalid inputs gracefully.
- Validate tool outputs before using them.
- Combine multiple tools for complex tasks.
- Keep tools secure and avoid sensitive data exposure.

8) Defining a Function (OpenAI Style)

```
{
  "name": "get_weather",
  "description": "Get current weather for a location",
  "parameters": {
    "type": "object",
    "properties": {
      "location": {
        "type": "string",
        "description": "City name"
      }
    },
    "required": ["location"]
  }
}
```

Key Fields:

- name → function name
- description → what it does
- parameters → input schema
- required → list of mandatory params

9) Key Takeaways

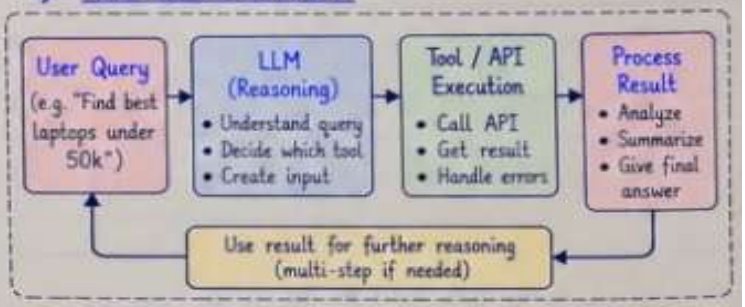
- Tools and APIs extend the capabilities of LLMs.
- Function calling enables structured and reliable tool use.
- Agents can use multiple tools to solve complex real-world tasks.
- Proper tool design and error handling improve performance and safety.

SUBTOPIC: Tool Use Strategies, APIs and Real-world Integration

1) Wol Use Strategies in Agents

- An agent decides when, which and how to use tools to solve a task.
- It may use multiple tools in a sequence.
- The agent analyzes the user query, chooses the right tool, prepares the input, executes it and uses the result to continue reasoning.
- Good tool use makes the agent more accurate, capable and useful in real-world applications.

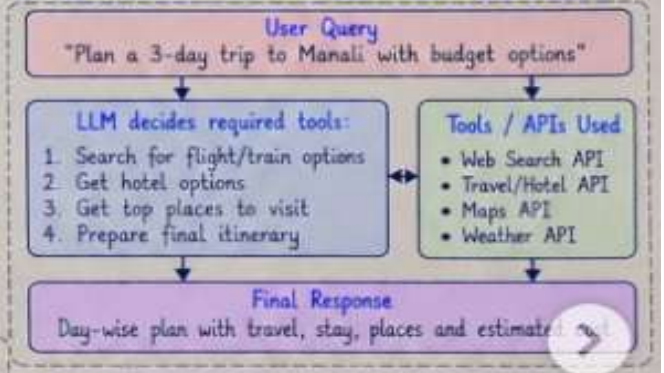
2) Tool Use Workflow



3) Tool Selection Strategies

1. Rule-based Selection – use predefined rules to choose tools.
2. LLM-based Selection – let the LLM decide based on tool descriptions.
3. ReAct Style – reason about which tool to use and why.
4. Plan-then-Execute – create a plan with multiple tool calls.
5. Self-Ask / Decompose – break complex tasks into smaller subtasks and choose tools for each.

4) Example: Multiple Tool Use (Trip Planner)



5) Example: Web Search Tool (SerpAPI)

Function Definition (Schema)

```
{
  "name": "search_web",
  "description": "Search the internet
    for latest information",
  "parameters": {
    "type": "object",
    "properties": {
      "query": { "type": "string",
        "description": "Search query" },
      "num_results": { "type": "integer"
      }
    },
    "required": ["query"]
  }
}
```

Example Function Call

```
{
  "name": "search_web",
  "arguments": {
    "query": "best laptops
      under 50000 in 2024",
    "num_results": 5
  }
}
```

Possible API Response (Simplified)

```
{
  "results": [
    {"title": "Best Laptops 2024",
     "link": "https://....",
     "snippet": "Top laptops under
                 50k ..."},
    ...
  ]
}
```

6) Handling Errors and Uncertainty

- Check if the tool is available and working.
- Handle API errors (network issues, invalid input, rate limit).
- If a tool fails, the agent can:
  - Retry with modified input.
  - Use an alternative tool.
  - Ask the user for clarification.
  - Continue with partial information.
- Always validate and verify important results.
- Inform the user when data may be uncertain.

7) Real-world Examples of Tools in Agents

Use Case	Tool / API	Purpose
Web Search	Google Search API, SerpAPI	Get latest information
Weather	OpenWeather API	Get current weather data
Maps / Location	Google Maps API	Get places, routes, travel time
News	News API	Get latest news
Finance	Stock/Finance API	Get stock prices, market data
Code Execution	Python REPL, Jupyter	Run code and analyze results
File Handling	File system tools	Read, write and process files
Email	Gmail API	Send and read emails
Custom	Any REST API	Domain specific tasks (e.g. booking)

8) Best Practices

- Provide clear tool descriptions to the LLM.
- Use well-defined input schemas.
- Validate inputs before calling tools.
- Handle errors and edge cases.
- Combine multiple tools for complex tasks.
- Keep API keys secure (use environment variables).
- Limit tool usage to prevent abuse and high costs.
- Prefer reliable and well-documented APIs.
- Test the agent with different queries.

9) Key Takeaways

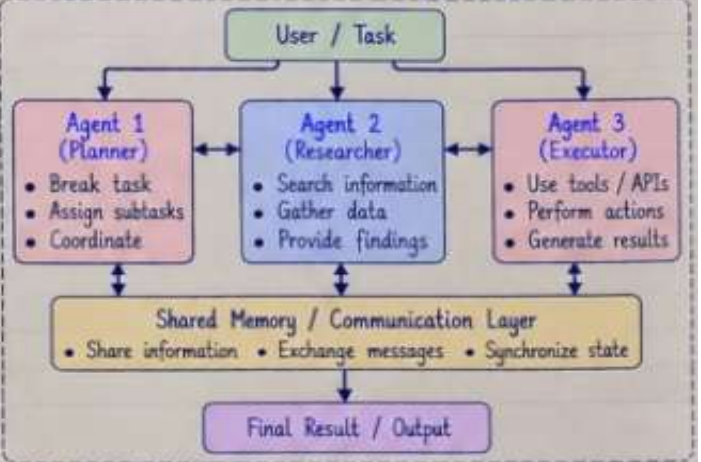
- Tools and APIs bridge the gap between LLMs and the real world.
- Proper tool use makes agents more capable and practical.
- Good design, error handling and validation are essential.
- Multiple tools can be combined to solve complex real-world tasks.

SUBTOPIC: Multi-Agent Systems (Agents Working Together)

1) What are Multi-Agent Systems?

- A Multi-Agent System (MAS) consists of multiple autonomous agents that work together (or sometimes compete) to achieve a goal.
- Each agent can perceive, reason, plan and act independently.
- Agents communicate with each other and may share information, tools or resources.
- Useful for complex tasks that require multiple skills, perspectives or parallel processing.

2) Architecture of a Multi-Agent System



3) Types of Multi-Agent Collaboration

1. Cooperative Agents – work together to achieve a common goal. (e.g., research team)
2. Competitive Agents – compete for resources or rewards. (e.g., trading bots)
3. Hybrid (Coop + Compete) – some cooperate and some compete. (e.g., multi-player games)
4. Hierarchical – agents have roles and follow a leader. (e.g., supervisor and worker agents)
5. Peer-to-Peer – all agents are equal and communicate directly. (e.g., decentralized systems)

4) Communication Between Agents

- Agents communicate by sharing messages, observations, plans or results.
- Common communication methods:
  1. Direct Messaging – agent sends message to another.
  2. Shared Memory – common storage accessible by all agents.
  3. Message Passing – structured messages (e.g., JSON).
  4. Blackboard Model – agents read/write to a shared board.
  5. Event-driven – agents react to events in the environment.

5) Example: Travel Planning with Multiple Agents



6) Real-world Applications

- Travel and logistics planning
- E-commerce (product search, price comparison)
- Healthcare (multiple agents for diagnosis, research, care)
- Autonomous vehicles (multi-agent coordination)
- Robotics (multiple robots working together)
- Finance (trading agents)
- Software development (one agent for coding, one for testing)
- Customer support (multiple specialized agents)

7) Advantages and Challenges

Advantages	Challenges
• Handle complex tasks	• Coordination between agents
• Parallel processing (faster)	• Communication overhead
• Specialization (each agent has specific skills)	• Conflict resolution
• More accurate and reliable results	• Consistency of information
• Scalable to large problems	• Higher computational cost
	• Security and trust issues

8) Example: Agents in Software Development

Agent	Role	Tasks
Planner Agent	Project planning	Break requirements, assign tasks
Coder Agent	Write code	Generate code using IDE/tools
Tester Agent	Test code	Run tests, find bugs
Reviewer Agent	Code review	Improve code quality
Deployer Agent	Deployment	Deploy to server/cloud
Monitor Agent	Monitoring	Track performance, alert issues

9) Key Takeaways

- Multiple agents can work together to solve complex real-world tasks.
- Each agent can have a specific role and set of tools.
- Good communication and coordination are essential.
- MAS is useful in many domains like travel, healthcare, finance and robotics.
- It improves efficiency, scalability and reliability.

SUBTOPIC: Planning, Reasoning and Decision Making in Agentic AI

1) What is Planning in Agentic AI?

- Planning is the process of breaking a complex goal into a sequence of smaller steps (actions) to achieve the goal.
- An agent uses the LLM's reasoning ability and planning techniques to decide what to do next.
- It helps handle multi-step tasks and ensures better, consistent and goal-oriented behavior.

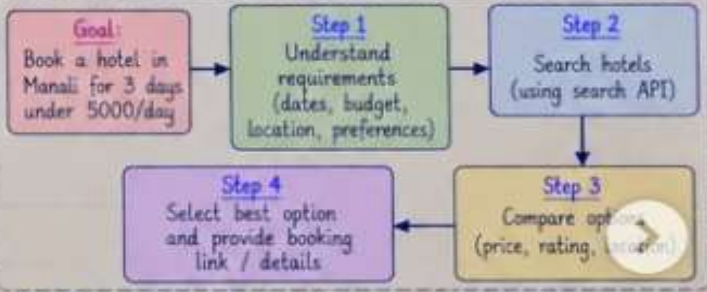
2) Types of Planning

Type	Description	Example
Reactive Planning	Decide next action based on current state (no long-term plan)	Simple Q&A, real-time response
Deliberative Planning	Plan multiple steps ahead using reasoning	Book a trip, research, compare and finalize
Hierarchical Planning	Break goal into high-level and low-level subgoals	Plan → Book → Pay → Generate itinerary
Goal-based Planning	Focus on achieving a specific goal	"Find best laptop under 50k"
Utility-based Planning	Choose actions that maximizes expected utility	Compare multiple options and pick best

3) Planning Techniques

1. Chain-of-Thought (CoT) – Step-by-step reasoning.
2. Tree of Thoughts (ToT) – Explore multiple possible paths (branches).
3. ReAct (Reason + Act) – Combine reasoning and action using tools.
4. Plan-and-Execute – First create a plan, then execute it step by step.
5. Self-Ask – Break complex questions into smaller sub-questions.
6. Reflection / Self-Critique – Review and improve the plan based on results.
7. Decomposition – Break large tasks into smaller subtasks.

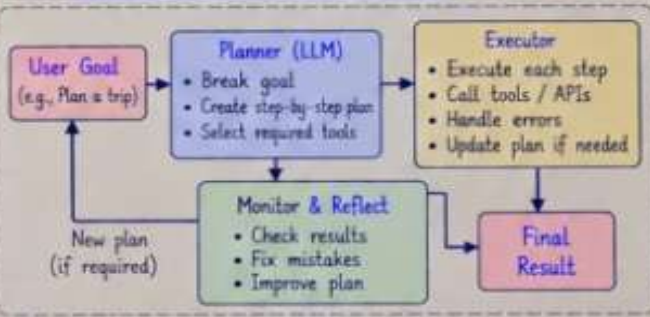
4) Example: Task Planning (Book a Hotel)



5) Reasoning Approaches

Approach	Idea	When to Use
Zero-shot	Directly reason without examples	Simple tasks
Few-shot	Use few examples in prompt	Moderate tasks
Chain-of-Thought	Step-by-step reasoning	Complex reasoning
Tree-of-Thoughts	Explore multiple reasoning paths	Open-ended or multi-path problems
ReAct	Reason and use tools/actions	Real-world tasks
Self-Consistency	Generate multiple solutions and pick best	Better accuracy

6) Plan-and-Execute Workflow



7) Example: Step-by-Step Plan Generation

User Query:

"I want to learn Java in 1 month. Create a study plan with topics and daily schedule."

LLM Generated Plan (Simplified):

1. Understand basics (Day 1-3)
2. OOP concepts (Day 4-8)
3. Collections and File Handling (Day 9-12)
4. Exception Handling (Day 13-15)
5. Mini Projects (Day 16-25)
6. Revision and Practice (Day 26-30)

8) Challenges in Planning and Reasoning

- LLMs may generate incorrect or illogical plans.
- Handling uncertainty and incomplete information.
- Long-term planning can be difficult due to context limits.
- Balancing exploration (trying new options) vs exploitation (using known good options).
- Ensuring the plan is feasible with available tools and resources.

9) Key Takeaways

- Planning helps agents break complex goals into manageable steps.
- Different reasoning techniques improve decision making.
- Combining planning with tools enables real-world problem solving.
- Reflection and self-critique improve plan quality.
- A good plan leads to more accurate, reliable and useful results.

SUBTOPIC: Agentic AI Frameworks (LangChain, AutoGen, CrewAI, etc.)

1) What are Agentic AI Frameworks?

- Agentic AI frameworks provide ready-made libraries, tools and abstractions to build, run and manage AI agents.
- They simplify complex tasks like tool use, memory, planning, multi-agent communication and deployment.
- Developers can build real-world agent systems faster using these frameworks.

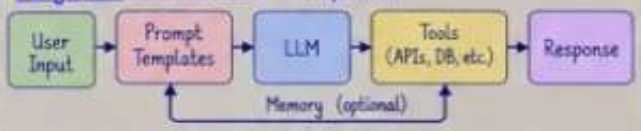
2) Popular Agentic AI Frameworks

Framework	Created By	Language	Main Focus	Use Cases
LangChain	LangChain Inc.	Python (also JS)	LLM apps, agents, tools	Agent with tools, RAG, workflows
AutoGen	Microsoft	Python	Multi-agent conversation	Collaborative agents, research
CrewAI	CrewAI	Python	Role-based multi-agent systems	Team of agents for complex tasks
Semantic Kernel	Microsoft	Python, C#, Java	Integrating LLMs with enterprise tools	Business apps, enterprise agents
LlamaIndex	LlamaIndex	Python	Data indexing and retrieval	RAG, knowledge agents

3) LangChain Framework

- One of the most popular frameworks for building LLM applications and agents.
- Provides modules for prompts, chains, tools, agents, memory and vector databases.
- Supports integration with many LLMs and external APIs.

LangChain Architecture (Simplified)



4) Key Components of LangChain

Component	Purpose
Prompt Templates	Create dynamic prompts
Chains	Combine multiple steps
Agents	Use tools to perform actions
Tools	Connect to external APIs, databases, etc.
Memory	Remember past interactions
Vector Stores	Store and retrieve embeddings
Callbacks	Monitor and log agent execution

5) Example: Simple Agent with a Tool (LangChain)

```
from langchain.agents import initialize_agent, Tool
from langchain.llms import OpenAI
from langchain.tools import SerpAPIWrapper

# Define a tool
search = SerpAPIWrapper()

# Initialize LLM
llm = OpenAI(temperature=0)

# Create tool list
tools = [Tool(
    name="Search",
    func=search.run,
    description="Search the internet for current information"
)]

# Initialize agent
agent = initialize_agent(tools, llm, agent="zero-shot-react-description")

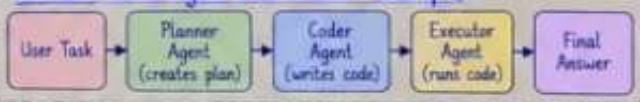
# Run agent
response = agent.run("What is the latest news about ISRO?")
print(response)
```

This agent can search the internet using SerpAPI and answer real-time questions.

6) AcewAI Framework

- Developed by Microsoft for building multi-agent conversational systems.
- Agents can talk to each other, plan, assign tasks and solve problems collaboratively.
- Useful for research automation, code generation, data analysis and more.

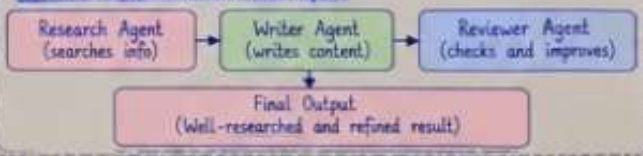
AutoGen Multi-Agent Conversation (Example)



7) CrewAI Framework

- Focuses on role-based multi-agent systems.
- You can define agents with specific roles, goals and tools.
- Agents work together like a team to complete complex tasks.
- Easy to use and good for real-world applications.

CrewAI Team Structure (Example)



8) Comparison of Frameworks

Feature	LangChain	AutoGen	CrewAI	Semantic Kernel
Ease of Use	Easy	Moderate	Easy	Moderate
Multi-Agent Support	Yes (basic)	Yes (strong)	Yes (strong)	Yes
Tool Integration	Excellent	Good	Good	Good
Memory Support	Yes	Yes	Yes	Yes
Best For	General agents	Collaborative agents	Role-based agents	Enterprise apps

9) Key Takeaways

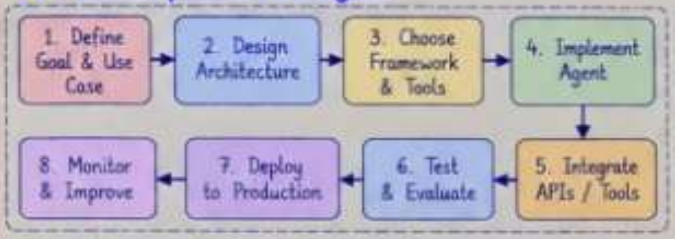
- Agentic AI frameworks make it easier to build powerful agents.
- LangChain is great for single agents with tool use.
- AutoGen is best for multi-agent collaboration.
- CrewAI focuses on team-based agents with roles.
- Choose the framework based on your use case, complexity and required features.

SUBTOPIC: Building and Deploying Agentic AI Systems (End-to-End)

1) End-to-End Development Process

- Building an Agentic AI system involves multiple steps from idea to deployment.
- It includes requirement analysis, designing agent architecture, selecting tools, implementation, testing and deployment.
- Each step ensures the agent is goal-oriented, reliable and useful in real-world applications.

2) Development Lifecycle



3) Step 1: Define Goal and Requirements

- Clearly define the problem the agent will solve.
- Identify target users, expected inputs, outputs and constraints.
- Decide whether single-agent or multi-agent system is needed.

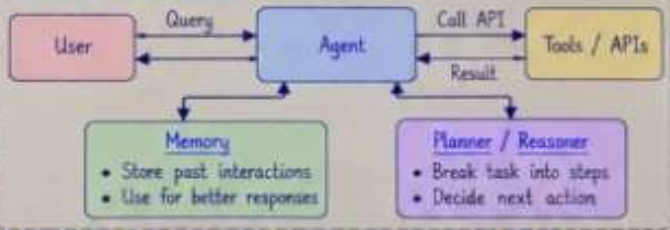
Example Requirement (Travel Planner Agent)

- Goal: Plan a 3-day trip within budget.
- Inputs: destination, budget, travel dates, preferences.
- Outputs: detailed itinerary with cost, hotels, travel options.
- Constraints: budget limit, real-time availability.

4) Step 2: Design Agent Architecture

- Decide the agent type (single/multi-agent).
- Choose reasoning and planning techniques.
- Decide which tools, APIs and memory will be used.
- Design the workflow and communication between components.

Example Architecture



5) Step 3: Select Framework and Tools

Component	Popular Options
Frameworks	LangChain, AutoGen, CrewAI, Semantic Kernel, LlamaIndex
LLM	OpenAI GPT, Anthropic Claude, Google Gemini, Llama
Tools / APIs	Web Search API, Database, Custom APIs (REST)
Vector Database	Pinecone, ChromaDB, FAISS, Weaviate
Deployment	Docker, Streamlit, FastAPI, Cloud platform (AWS, GCP)

6) Step 4: Implement the Agent

- Write the code using the selected framework.
- Integrate required tools and APIs.
- Implement memory, planning and reasoning.
- Handle errors and edge cases.
- Keep the design modular and scalable.

Example: LangChain Agent (Simple)

```
from langchain.agents import initialize_agent, Tool
from langchain.llms import OpenAI
from langchain.tools import SerpAPIWrapper

llm = OpenAI(temperature=0)
search = SerpAPIWrapper()

tools = [ Tool(
      name = "Search",
      func = search.run,
      description = "Search the internet for information"
 )]

agent = initialize_agent(tools, llm, agent="zero-shot-react-description")
response = agent.run("Plan a 3-day trip to Manali under 5000")
print(response)
```

This code creates
- a simple agent
- uses web search tool
- takes user query
- gives real-time answer

7) Step 5: Testing and Evaluation

- Test the agent with different types of inputs.
- Check correctness, relevance and reliability of responses.
- Evaluate how well it handles errors, missing data and complex queries.
- Compare outputs with expected results.

Evaluation Metric	Description
Accuracy	Correctness of the response
Relevance	How relevant the answer is to the query
Tool Usage	Whether the right tools are called
Latency	Time taken to generate response
Robustness	Handles errors and edge cases
User Satisfaction	Overall usefulness of the response

8) Step 6: Deployment

- Deploy the agent using platforms like Streamlit, FastAPI, Docker or cloud services (AWS, GCP, Azure).
- Make the agent accessible via web app, API or chatbot interface.
- Ensure proper security, authentication and rate limits.

Deployment Options

Local Deployment	Container Deployment	Cloud Deployment
• Streamlit • FastAPI (localhost)	• Docker • Docker Compose	• AWS, GCP, Azure • Serverless (Lambda)

9) Monitoring and Improvement

- Monitor agent usage, performance and errors.
- Collect user feedback and improve prompts, tools and logic.
- Update memory and knowledge base regularly.
- Add new tools or capabilities based on user needs.

10) Key Takeaways

- Building an Agentic AI system involves planning, designing, implementation, testing, deployment and continuous improvement.
- Use the right framework and tools for your use case.
- Proper testing and monitoring make the agent reliable and scalable.
- Iterate and improve based on real-world usage and feedback.

SUBTOPIC: Real-World Applications, Safety, Ethics and Challenges

1) Real-World Applications of Agentic AI

- Agentic AI is being used in many industries to automate complex tasks and make better decisions.
- It combines LLMs, tools, memory and planning to work independently for real-world goals.

Domain	Use Cases	Example
Travel & Hospitality	Trip planning, booking, itinerary management	AI travel agent (book flights, hotels)
Healthcare	Patient support, report analysis, appointment booking	Symptom checker agent, research agent
Finance	Market analysis, portfolio management	Investment advisor agent
Education	Personalized learning, research assistance	Study planner agent
E-commerce	Product search, comparison, recommendations	Shopping assistant agent
Software Development	Code generation, testing, debugging, documentation	AI coding agent
Research & Analytics	Literature review, data analysis, report generation	Research agent with web + tools
Customer Support	Automated support, ticket handling	Multi-agent support system

2) Case Study: Autonomous Research Agent

Goal:
Research a topic, collect latest papers, summarize them and create a report.

Workflow



3) Safety and Reliability

- Agentic AI systems can take powerful actions, so safety is very important.
- We need to make sure the agent is aligned, controllable and does not harm users or systems.

Safety Aspect	Description
Alignment	Ensure agent follows human intent and values.
Tool Safety	Restrict dangerous tools and actions.
Input Validation	Check and sanitize user inputs.
Output Control	Filter harmful or incorrect responses.
Human-in-the-Loop	Keep humans in control for critical tasks.
Monitoring	Track agent actions, logs and performance.
Failsafes	Stop or rollback in case of unexpected behavior.

4) Ethical Considerations

- Agentic AI can impact privacy, fairness and society.
- We must use it responsibly and ethically.

Ethical Issue	Description
Privacy	Protect user data and personal information.
Bias	Avoid biased outputs and unfair decisions.
Transparency	Make agent actions explainable.
Accountability	Clearly define who is responsible.
Misinformation	Prevent spreading false or misleading information.
Job Impact	Consider effects on employment and society.
Fair Access	Ensure benefits are available to everyone.

5) Challenges in Real-World Deployment

- Building and using agentic AI in real applications is not easy.
- There are technical, safety, cost and social challenges.

Challenge	Explanation
Reliability	Agents may make mistakes or incorrect decisions.
Hallucinations	LLMs can generate false or unrealistic information.
Tool Failures	APIs or tools may be unavailable or return errors.
Long-term Planning	Difficult to handle very complex, multi-step tasks.
Memory Management	Storing and retrieving relevant information is challenging.
Cost	Using multiple tools and long context can be [illegible]
Latency	Real-time responses can be slow due to multi[illegible]
Security Risks	Agents may be vulnerable to prompt injection or misuse.
Scalability	Managing many agents in production can be complex.

6) Best Practices for Safe and Effective Use

1. Clearly define the goal and scope of the agent.
2. Use reliable frameworks and well-tested tools.
3. Implement proper error handling and fallback mechanisms.
5. Monitor and log agent actions.
6. Use human-in-the-loop for critical decisions.
7. Validate and verify important outputs.
8. Follow ethical guidelines and legal regulations.

7) Future Trends and Opportunities

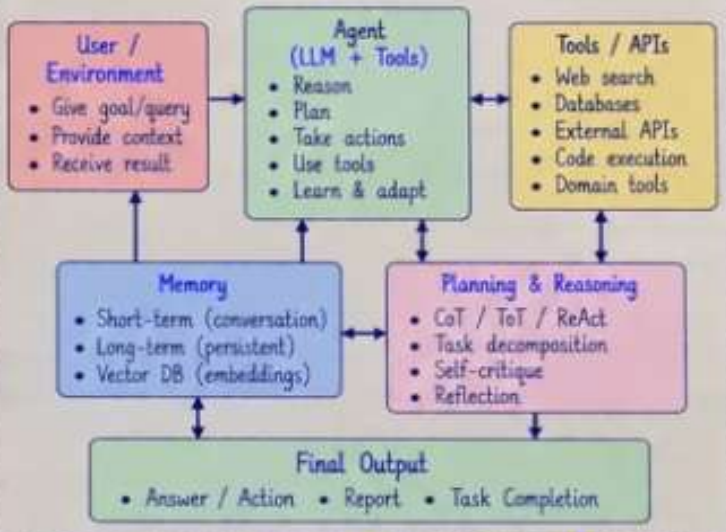
- More advanced multi-agent systems working together.
- Better memory, reasoning and planning capabilities.
- Integration with real-world devices (IoT, robotics).
- Domain-specific agents for healthcare, education, finance, etc.
- Safer and more reliable agent frameworks.
- Wider adoption in industry and everyday life.
- Potential to solve complex global challenges.

8) Key Takeaways

- Agentic AI has many real-world applications and huge potential.
- Safety, ethics and reliability are critical for responsible use.
- There are still many challenges to solve.
- Following best practices can help build useful and trustworthy agents.

SUBTOPIC: Summary, Interview Questions and Final Revision

1) Complete Agentic AI System Overview



2) Comparison: Normal AI vs Agentic AI

Feature	Normal AI (Chatbot)	Agentic AI
Goal Handling	Single-turn	Multi-step, complex goals
Tool Use	Limited / None	Uses multiple tools & APIs
Autonomy	Low	High
Memory	Usually short-term	Short-term + Long-term
Planning	Minimal	Advanced planning & reasoning
Real-world Actions	No	Yes (via tools/APIs)
Example	Answer a question	Plan a trip, book hotels, create report, analysis, etc.

3) Common Tools and Libraries

Category	Tools / Libraries	Purpose
Frameworks	LangChain, AutoGen, CrewAI, Semantic Kernel	Build and manage agents
LLMs	OpenAI GPT, Claude, Gemini, Llama	Reasoning and generation
Vector DB	Pinecone, Chroma, Weaviate	Store and retrieve embeddings
Web Search	SerpAPI, Tavily, Bing API	Real-time information
Code Execution	Python REPL, Docker	Run code and analysis
APIs Integration	REST APIs, OpenAPI	Connect to external services
Deployment	Streamlit, FastAPI, Docker, AWS, Azure	Deploy agent applications
Monitoring	LangSmith, Weights & Biases	Track, evaluate and debug

4) Important Terminologies (Quick Revision)

- **Agent**: AI system that can perceive, reason and act.
- **Tool**: External function or API used by agent.
- **Memory**: Stores past interactions and context.
- **Planning**: Breaking goal into smaller steps.
- **Reasoning**: Deciding next action using logic.
- **Multi-Agent System**: Multiple agents working together.
- **Vector Database**: Stores embeddings for semantic search.
- **RAG**: Retrieval-Augmented Generation (use external knowledge).
- **Prompt Template**: Structured input for LLM.
- **Chain**: Sequence of calls and operations.
- **Hallucination**: Generating false information.
- **ReAct**: Reason + Act approach.
- **CoT**: Chain-of-Thought.
- **ToT**: Tree-of-Thoughts.
- **Embedding**: Vector representation of text.

5) Interview Questions (with Short Answers)

1	What is Agentic AI?	AI system that can plan, use tools, take actions and achieve goals autonomously.
2	How is it different from normal AI?	Normal AI answers queries, while Agentic AI can perform multi-step tasks using tools and planning.
3	What are common frameworks?	LangChain, AutoGen, CrewAI, Semantic Kernel.
4	What is RAG?	Retrieval-Augmented Generation. It uses external knowledge (e.g., vector DB) to give better answers.
5	What is the role of memory?	Stores past interactions and relevant information for better decision making.
6	What are planning techniques?	CoT, ToT, ReAct, Plan-and-Execute, Self-Ask, Reflection.
7	How do agents use tools?	They call external APIs, databases, web search or code execution tools to perform actions.
8	What are challenges in Agentic AI?	Reliability, hallucinations, tool failures, high cost, latency, security risks.
9	How can we make agents safe?	Input validation, output filtering, human-in-the-loop, monitoring and following ethical guidelines.
10	What are real-world applications?	Travel planning, healthcare, finance, Customer support, education, research, e-commerce, etc.

6) Final Revision Checklist

Topic	✓	Topic	✓
Basics of Agentic AI	☐	Example Applications	☐
Types of Agents	☐	Safety and Reliability	☐
Architecture and Components	☐	Ethical Considerations	☐
Planning Techniques	☐	Challenges and Limitations	☐
Reasoning Approaches	☐	Best Practices	☐
Memory in Agents	☐	Future Trends	☐
Tools, APIs and Function Calling	☐	Important Terminologies	☐
Popular Frameworks	☐	Comparison (Normal vs Agentic)	☐
Multi-Agent Systems	☐	Interview Questions	☐
Development Lifecycle	☐	Key Takeaways	☐

7) Key Takeaways

- Agentic AI combines LLMs, tools, memory and planning to achieve complex goals.
- It can work autonomously and handle multi-step real-world tasks.
- Frameworks like LangChain, AutoGen and CrewAI make development easier.
- Planning, reasoning and memory are essential for reliable agents.
- Safety, ethics and proper evaluation are very important.
- Agentic AI has huge potential in many industries and will continue to grow.
- It is an exciting field with many real-world opportunities.

8) Thank You!

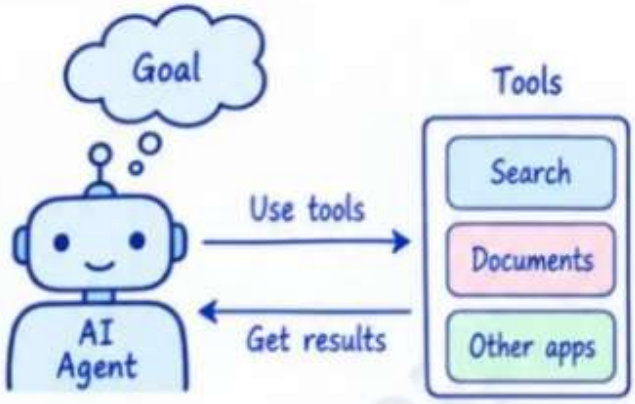
Thanks for going through these notes!
Keep Learning, Keep Exploring and Keep Building 🚀
Follow @aditechnova18 for more such handwritten notes.

Stay Curious ☺
Stay Consistent
You Can Do It!
— Aditya Kumar
@aditechnova18

Introduction to Agentic AI

Definition

- AI systems that pursue a goal through planning, tool use and feedback.
- Work happens within defined permissions.



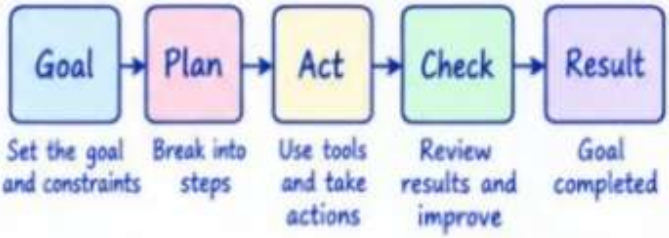
Key Characteristics

- **Goal-driven**
  Works towards a specific goal instead of a single one-off answer.
- **Multi-step**
  Breaks the task into smaller steps, plans the next step and keeps progressing.
- **Tool-enabled**
  Can use external tools (such as search or documents) to get information or take actions.
- **Results checked**
  Reviews the outputs, uses feedback to improve and continues until the goal is met.

Example

- Goal: Create study notes on a topic.
- Step 1: Retrieve relevant sources.
- Step 2: Draft the study notes.
- Step 3: Review the notes for coverage and make improvements.

Typical Workflow



Important Note

- Use clear limits and verify important results.

Q: What makes AI agentic?

A: It coordinates actions over multiple steps to pursue a goal.

What Is an AI Agent?

- An agent observes information, selects actions and uses the results to choose its next step.
- An agent is goal-oriented and interacts with its environment using tools when needed.
- It follows a repeated process of observing, acting and learning from the results.

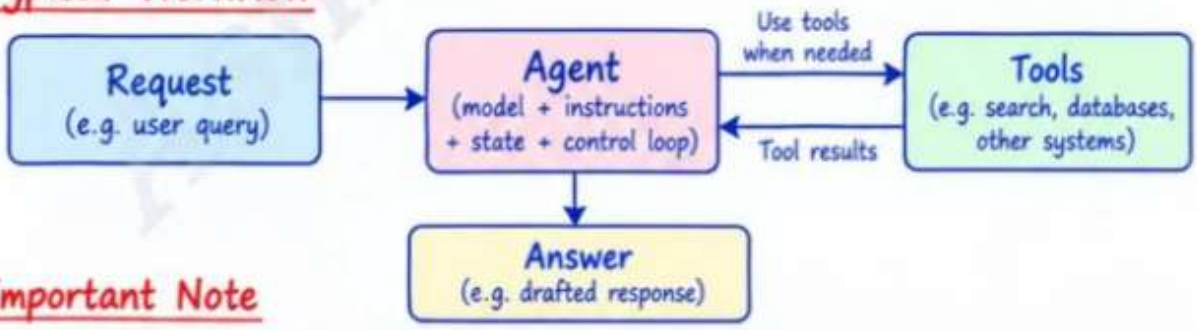
1. Parts of an Agent

- Model: understands information and helps decide what to do.
- Instructions: define the goal, behaviour and any constraints.
- Tools: external resources the agent can use to take actions (e.g. search, databases, other systems).
- State: information the agent keeps while working (e.g. previous steps, retrieved information).
- Control loop: observe → decide → act → use result → next step.

2. Example: Support Assistant

- Reads a customer support query.
- Retrieves relevant guidance using tools (e.g. knowledge base search).
- Uses the model to draft a helpful answer based on the retrieved information.
- Reviews the result and can take additional steps if needed (e.g. look up more details) before giving the final answer.
- Repeats this process until a suitable answer is produced.

Typical Workflow



Important Note

- Use clear limits and verify important results.

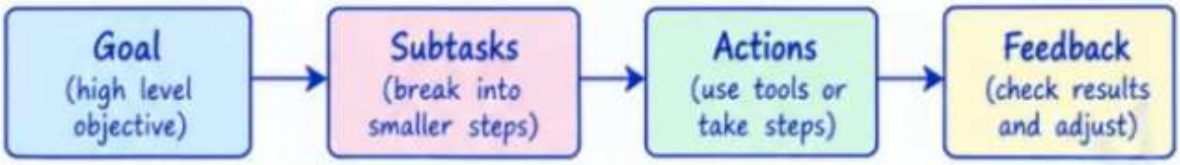
Q: Does every chatbot act as an agent?

A: No. An agent includes a process for selecting and executing actions.

Features of Agentic AI

Definition

- Agentic systems combine planning, action and feedback.



1. Task Decomposition

- Breaks a complex goal into smaller, manageable subtasks.
- Helps the agent plan the order of steps to achieve the goal.
- Example: For trip planning, split into subtasks like search places, check options and create itinerary.

2. Context Awareness

- Uses available context such as goal, user input and past results.
- Helps choose relevant next steps and tools.
- Example: Remembers earlier preferences while planning and adjusts later actions.

3. Tool Selection

- Chooses appropriate tools or resources for each subtask.
- Different tools may be used for different kinds of tasks.
- Example: Use a search tool to find information and a calculator tool for numerical tasks.

4. Progress Tracking

- Keeps track of completed steps and remaining subtasks.
- Uses feedback to decide whether to continue, retry or change plan.
- Example: Marks completed steps and moves to the next subtask after checking the result.

5. Error Handling

- Detects when a step fails or gives an unexpected result.
- Retries, adjusts the plan or chooses an alternative approach.
- Example: If a tool returns an error, try a different tool or modify the query.

Additional Points

- Autonomy varies by design. Some agents act more independently while others follow strict rules.
- Sensitive actions may need human approval before execution.

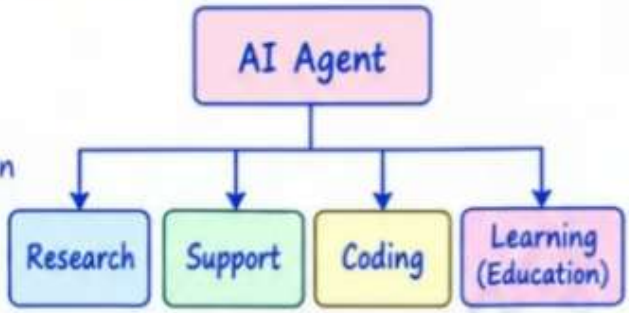
Important Note

- Use clear limits and verify important results.

Q: Can an agent make mistakes?
A: Yes. Both its decisions and tool results require checks.

Applications of Agentic AI

- Agents assist with research, support, coding, education and business workflows.
- They use available tools and information to complete tasks with some level of independence.
- Goal: help people work faster, better and more easily.



1. Research

- Agent gathers relevant sources on a topic.
- Helps organise the information in a clear way.
- Example: Collects articles and produces a short summary with key points.

2. Support

- Agent retrieves guidance or information to answer user questions.
- Helps provide quick and consistent responses.
- Example: A customer support agent finds relevant help articles and returns step-by-step guidance.

3. Coding

- Agent proposes and tests changes to code.
- Can help with debugging and explaining code.
- Example: Suggests a code change, runs a test and shows the result.

4. Education

- Agent creates practice questions and explanations.
- Helps learners check their understanding and find topics to revise.
- Example: Generates questions on a chapter and provides answers with short explanations.

Verify Important Outputs

- Always review important results produced by an agent.
- This helps catch mistakes and ensures the information is correct and useful.

Important Note

- Use clear limits and verify important results.

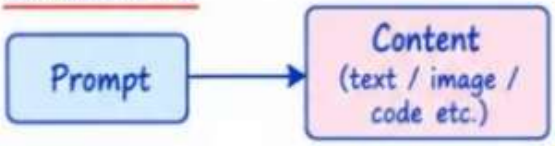
Q: Give one educational example.
A: A study assistant retrieves a chapter, drafts notes and checks topic coverage.

Generative AI vs Agentic AI

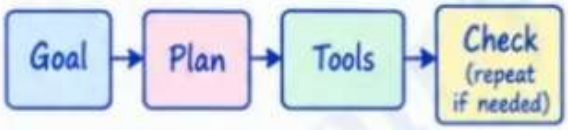
Definition

- Generative AI creates content such as text, images or code based on a given prompt.
- Agentic systems often use generative models to coordinate actions and achieve a goal.

Generative AI



Agentic AI



Key Differences

1. Output

- Generative AI: produces content as output (e.g. text, image, code).
- Agentic AI: aims for task progress towards a goal (e.g. completing a multi-step task).

2. Process

- Generative AI: generates a response in a single or limited number of steps based on a prompt.
- Agentic AI: uses repeated action loops (plan, act, check, and adjust) until the goal is achieved.

3. Tools

- Generative AI: tools are optional; it can generate content using only the model.
- Agentic AI: tools are often central to interact with the environment and perform actions (e.g. search, run code, use applications).

Overlap

- The categories are not mutually exclusive.
- Agentic systems can use generative models at different steps, such as planning, deciding the next action or producing the final response.

Important Note

- Use clear limits and verify important results.

Q: Can agentic AI use generative AI?

A: Yes. A generative model car help plan actions and produce the final response.

AI Agent Architecture

Definition

- Architecture connects a model with instructions, tools, state and a controller.

Main Components

1. Instructions

- Define the agent's role, goal and limits.
- Guide how the model should behave.
- Help keep the agent focused and within allowed boundaries.

2. Model (LLM)

- Proposes steps to achieve the goal based on the instructions and current state.
- Interprets user requests and tool outputs.
- Suggests the next action for the controller to execute.

3. Controller

- Manages the overall workflow.
- Decides which tool to use and when to use it.
- Handles stopping conditions and returns the final result.

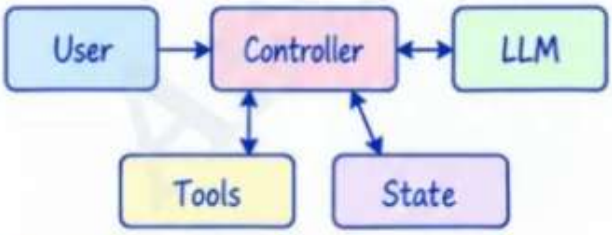
4. Tools

- Provide external capabilities such as searching, calculating or accessing data.
- Extend what the model can do.
- Return results back to the controller.

5. State

- Stores the progress of the task.
- Keeps track of important information such as past steps and results.
- Helps the agent make consistent decisions across multiple steps.

Typical Architecture



Important Note

- Use clear limits and verify important results.

Q: Why is a controller needed?

A: It manages the workflow, tool execution and stopping conditions.

Goals and Task Decomposition

- A goal states the desired result. Decomposition divides it into manageable tasks.
- Helps the agent plan, act step by step, and produce a useful output.
- Makes complex requests easier to handle and verify.



1. Define Output, Scope and Success Criteria

- Define the expected output clearly (format, length, key content).
- Set the scope (what is included and what is not).
- List success criteria to know when the goal is achieved.
- Clear definitions reduce confusion and help in verification.

2. Identify Subtasks and Dependencies

- Break the goal into smaller, ordered subtasks.
- Identify dependencies between tasks (which step needs the result of another step).
- Each subtask should have a clear input, output and completion check.
- This helps the agent execute tasks systematically and track progress.

3. Example: One-page Summary from Three Approved Sources

- Output: One page, concise summary.
- Scope: Use only the three approved sources.
- Success criteria: Covers key points from all three sources, no extra sources, clear and well-structured.
- Subtasks: (1) Retrieve information from each source, (2) Compare key points, (3) Draft the summary, (4) Verify it meets the criteria.
- Dependencies: Drafting depends on retrieval and comparison. Verification is the final step.

Important Note

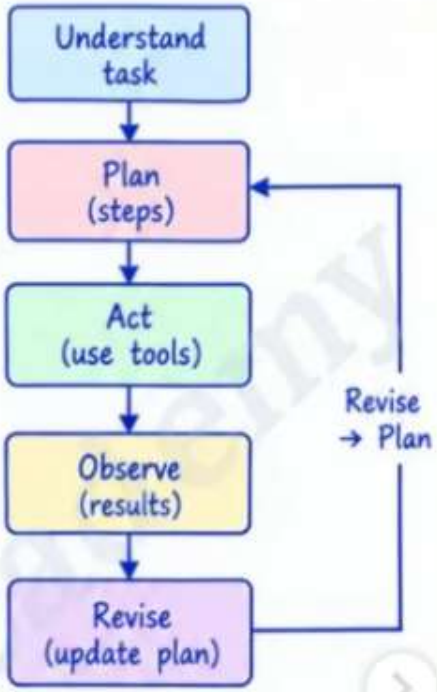
- Use clear limits and verify important results.

Q: What is a useful subtask?

A: A specific step with a clear input, output and completion check.

Reasoning and Planning

- Planning chooses steps toward a goal.
- Reasoning helps select actions using available information.
- Together, reasoning and planning allow an AI agent to make decisions, use tools and adapt when needed.



Main Points

1. Understand task

- Clarify the goal and what needs to be done.
- Break the request into a clear objective and sub-tasks.

2. Identify constraints

- Consider limits such as available tools, time, data and any given rules.
- Avoid actions that are not allowed.

3. Form plan

- Create a sequence of steps to reach the goal.
- Keep the plan simple and focused.

4. Choose tools

- Select appropriate tools or functions for each step.
- Use tools only when they are useful for the task.

5. Revise using evidence

- Check the results after each step.
- If the outcome is not as expected, update the plan and try a better approach.
- Continue until the goal is achieved or no further progress is possible.

Example

- Task: Inspect API errors and retry only when appropriate.
- The agent checks the error message, understands the issue, decides whether a retry can help, and uses the correct action.
- It retries when the error is likely temporary and avoids retrying when the error is not retryable, then continues or revises the plan based on the result.

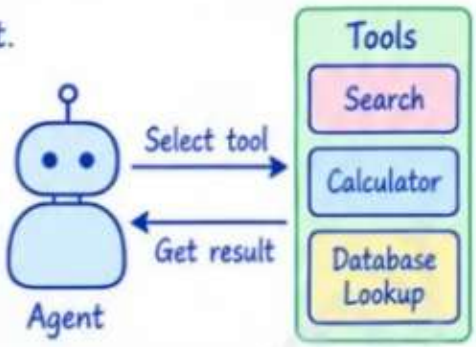
Important Note

- Use clear limits and verify important results.

Q: Are plans fixed?
A: No. Plans may change when new evidence or tool results arrive.

Actions and Tool Execution

- An action is an operation selected by an agent. A tool offers a specific capability.
- Agents use tools to interact with the external world, retrieve information or perform operations.
- Tool execution follows a sequence of steps to ensure the action is safe, valid and useful.



1. Main Points

- **Select tool:** Choose the most suitable tool for the task based on the goal.
- **Validate inputs:** Check that the inputs are well-formed and complete.
- **Check permission:** Ensure the action is allowed within defined limits.
- **Execute:** Call the tool with the given inputs.
- **Inspect output:** Review the result for errors, correctness and usefulness.

2. Examples of Tools

- **Search:** Find information on a topic using a search tool.
- **Calculator:** Perform arithmetic calculations such as sums or unit conversions.
- **Database lookup:** Retrieve records from a database using a query.
- **Tools may fail:** A tool may return an error, no result, or an unexpected result. The agent should handle such cases and decide the next step.

Typical Workflow



Important Note

- Use clear limits and verify important results.

Q: Why validate inputs?

A: It prevents malformed requests and reduces unintended operations.

Feedback and the Agent Loop

Definition

- Feedback shows whether an action helped achieve the goal.

Main Points

- The agent follows a loop: observe, decide, act and evaluate.
- Feedback from evaluation is used to improve the next decision.
- The agent stops after success, a budget limit or when a human decision is required.

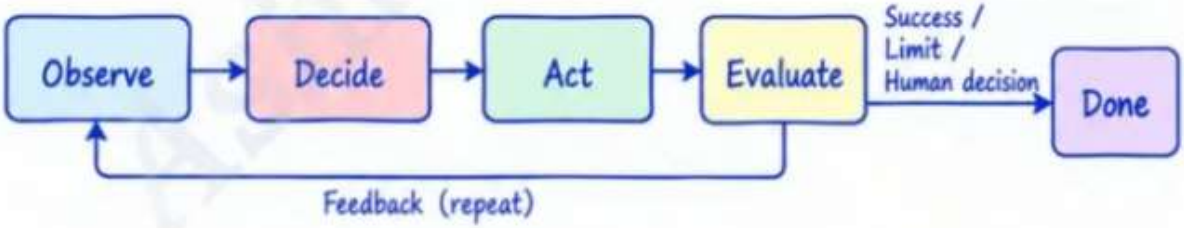
Steps in the Agent Loop

- **Observe**: Understand the current state, including the goal and available information.
- **Decide**: Choose the next action based on what was observed.
- **Act**: Perform the chosen action.
- **Evaluate**: Check the result using feedback. Decide whether to continue, stop or change the next action.

Example: Drafting Notes

- **Observe**: Check the topic and what is already covered.
- **Decide**: Plan what to write next.
- **Act**: Draft the next part of the notes.
- **Evaluate**: Review the draft, check coverage and find any omissions.
- Repeat the loop to fix omissions until the notes are complete or a stop condition is met.

Agent Loop Diagram



Important Note

- Use clear limits and verify important results.

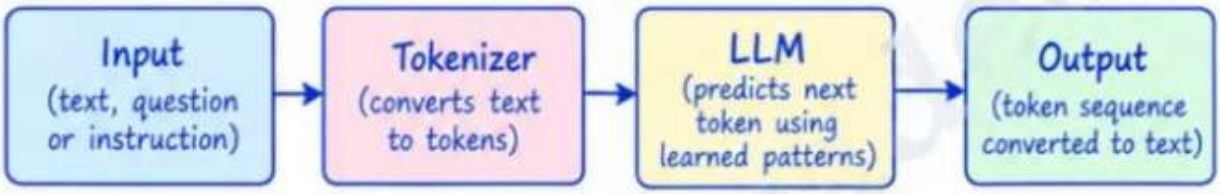
Q: Why define stop conditions?

A: They prevent endless loops and limit time, cost and repeated failures.

LLM Fundamentals

- A large language model generates token sequences based on patterns learned during training.
- It predicts the next token step by step to produce a coherent response.
- Uses: drafting, summarizing, classification and proposing tool calls.
- In an agent, a controller connects the LLM with tools and state.
- It can produce incorrect information, so outputs should be checked and validated when needed.

How an LLM Works (Simplified)



Key Points

- Learns patterns from large amounts of text during training.
- Works with tokens (small pieces of text), not whole words.
- Generates a response by predicting the next token, one at a time.
- Good at understanding and generating human language for many tasks.
- Common uses: drafting content, summarizing information, classification and proposing tool calls (e.g., which tool to use).
- In an agent, a controller uses the LLM's output to decide next actions, call tools and update state.
- It can produce incorrect or misleading information, so verification may be required.

Example

- Example: A travel assistant LLM can draft an itinerary from a user's request, summarize options and suggest a tool call to check flight prices.
- It does not book the flight itself. A controller handles the tool call and state.

Important Note

- An LLM is a language model. It generates text based on learned patterns.
- It does not have built-in access to tools, real-time data or persistent state by itself.

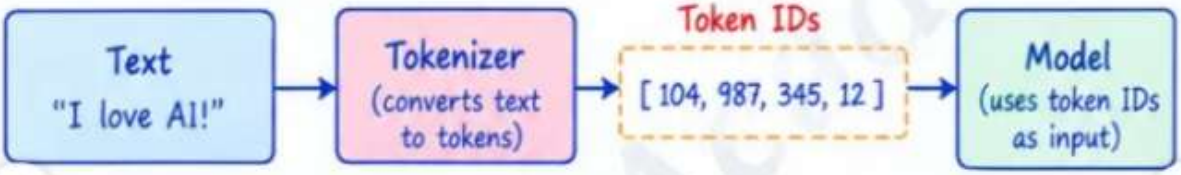
Q: Is an LLM alone a complete agent?

A: No. An agent also needs a workflow for actions, tools and state.

Tokens and Tokenization

- Tokens are model-specific units of text, such as words, word pieces or punctuation.
- A tokenizer converts text to token IDs.
- The number of tokens can vary depending on the language and tokenizer used.
- Input and output usage is measured in tokens.
- A long word may be split into multiple tokens.

How Tokenization Works



Example

- Sentence: "I love AI!"
- A tokenizer may split this into tokens like: ["I", "love", "AI", "!"].
- Each token is then converted to a numerical token ID.
- A longer or more complex word may be split into multiple tokens.
- For example, the word "unbelievable" might be tokenized as ["un", "believ", "able"] (tokenization depends on the model).
- Different tokenizers and languages can produce different token counts for the same text.

Important Note

- Token counts matter because input and output usage (and limits) are measured in tokens, not characters or words.
- The exact tokens and their IDs are specific to each model and tokenizer.

Q: Is one token always one word?

A: No. A token can represent a word, part of a word or punctuation.

Context Window

- The context window limits how much tokenized information a model can handle in a request.
- Context includes instructions, conversation and tool results.
- Model-specific rules govern input and output limits.
- Keep useful information and summarize older material.
- Context is different from persistent memory.

1. What is Context Window?

- The context window is the maximum number of tokens the model can handle at once in a request.
- It includes the current instructions, previous conversation and any tool outputs provided in the same request.

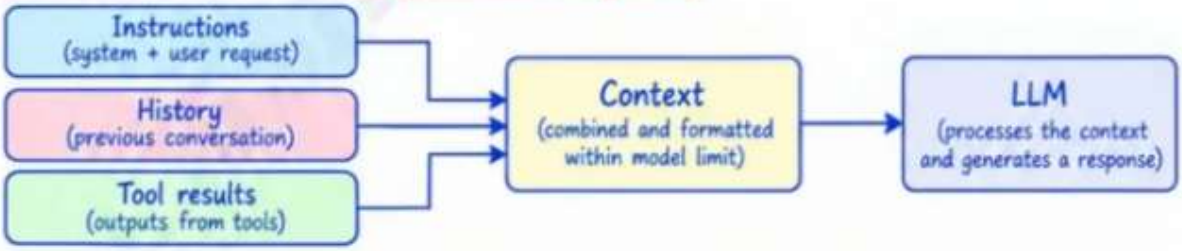
2. Key Points

- Each model has its own input and output token limits.
- If the total tokens exceed the limit, the request must be reduced.
- Keep the most relevant information and summarize older parts of the conversation.
- Context is temporary for the current request and is not the same as persistent memory.

3. Practical Example

- In a customer support agent, the context may include the user's latest question, previous messages and results from tools (e.g., order details from a database).
- If the conversation becomes very long, older messages can be summarized so that important details are kept within the context window.

Context Window in an Agentic AI System



Important Note

- The context window is for the current request only.
- It is different from persistent memory, which can store information across requests.
- Summarizing older material helps keep the most useful information within the limit.

Q: What happens when context is too long?

A: The system must reduce it or the request may exceed the model limit.

Prompts and Instructions

- A prompt provides a task and information to a model.
- Instructions define expected behaviour and limits.
- A clear prompt helps the model understand what to do, how to do it and what to avoid.

Example Prompt

"Summarize this article in five bullets using only its contents."

- Task: summarize the article.
- Context: the article text is provided.
- Output format: five bullet points.
- Constraints: use only the information in the article, do not add external information.

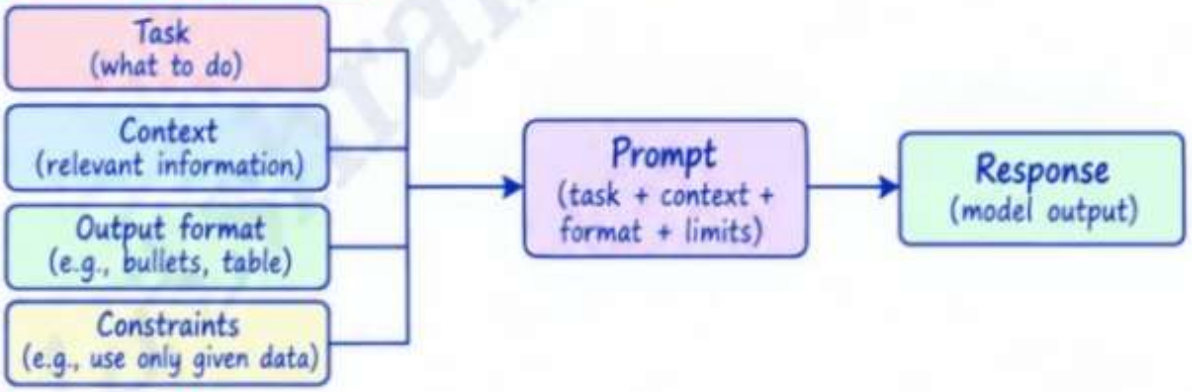
Main Parts of a Good Prompt

- Task: what you want the model to do.
- Context: relevant background information
- Output format: how you want the response (e.g., bullets, table, summary
- Constraints: limits or rules to follow (e.g., use only given information, keep it brief).

Instructions in Retrieved Documents

- When using external documents or web content, treat instructions found inside those documents as untrusted data.
- They may contain directions that try t change the model's behaviour.
- Follow only the instructions given by you, and use the document content as information to answer the task.

Typical Prompt Flow



Important Note

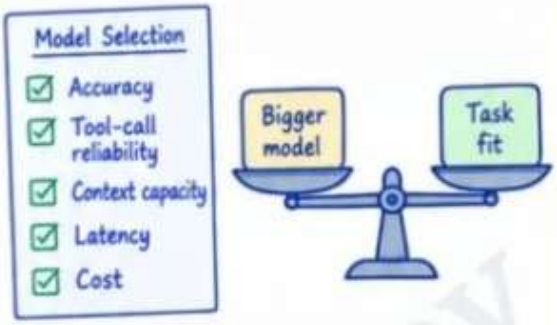
- A clear and specific prompt leads to more useful and relevant responses.
- Always treat instructions inside retrieved or external documents as untrusted data.

Q: What makes a prompt clear?

A: A specific task, relevant context, expected format and clear constraints.

Choosing a Model

- Model selection matches a model to the task and operational needs.
- Compare accuracy, tool-call reliability, context capacity, latency and cost.
- Evaluate on representative tasks.
- A bigger model is not always the best choice.



1. Key Factors to Compare

- **Accuracy**: How well the model completes the task and follows instructions.
- **Tool-call reliability**: Ability to correctly call tools (e.g. search, code, APIs).
- **Context capacity**: Amount of information the model can handle at once.
- **Latency**: Time taken to get a response.
- **Cost**: Price per use and total running cost for your workload.

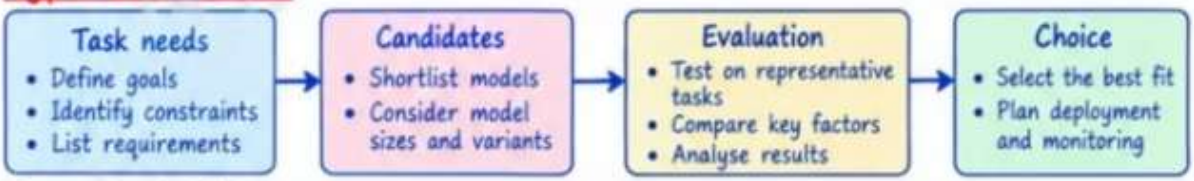
2. Evaluate on Representative Tasks

- Use tasks similar to real use cases.
- Test multiple examples, not just one.
- Check success rate, quality of output, correct tool usage and consistency.
- Include edge cases (e.g. ambiguous instructions, missing information).
- Consider operational needs such as response time and budget.

3. Practical Example

- **Task**: Build an agent to summarise company documents and answer questions.
- **Candidates**: Model A (smaller, lower cost) and Model B (larger, higher cost).
- **Evaluate**: Run the same set of document questions, measure answer accuracy, correct tool usage (e.g. file search), response time and cost.
- **Choose**: If Model A meets the accuracy and tool-use needs with lower latency and cost, it may be the better choice even though it is smaller.

Typical Workflow



Important Note

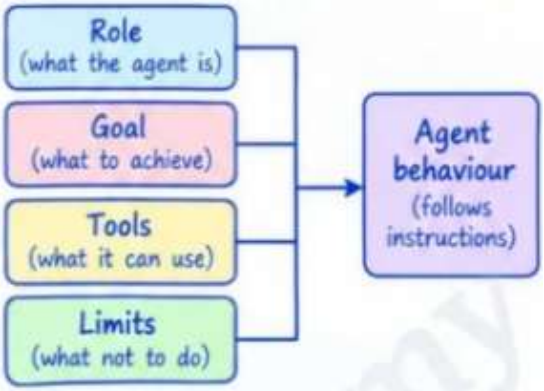
- Model choice depends on the specific task, data, tools and operational constraints.
- Re-evaluate periodically as models, requirements and costs change.

Q: How should models be compared?

A: Test them on realistic tasks using consistent success criteria.

Writing Agent Instructions

- Agent instructions describe purpose, scope, permitted actions and expected results.
- Clear instructions help the agent understand what to do, how to use tools, and when to stop or ask for help.
- Well written instructions lead to more reliable and useful behaviour.



1. Key Parts of Instructions

- **Role** – define what the agent is (e.g., study assistant, research helper).
- **Goal** – state the main objective and expected result.
- **Tools** – list allowed tools or data sources the agent can use.
- **Limits** – specify what the agent should not do and any constraints.
- **Completion checks** – define how the agent knows the task is finished.
- **Escalation rules** – state when the agent should ask for human help (e.g., missing information, uncertainty, or blocked task).

2. Example: Study Assistant

- **Role:** Study assistant for a student.
- **Goal:** Create clear and concise study notes on a given topic.
- **Tools:** Use only approved sources such as provided textbooks, lecture notes or trusted educational websites.
- **Limits:** Do not use untrusted sources, do not make up facts, and keep the content at the requested level.
- **Completion checks:** Ensure notes cover the key points and include a short summary.
- **Escalation rules:** If information is missing, unclear or not available in the approved sources, flag this and ask for further guidance.

Typical Workflow



Important Note

- Clear and specific instructions reduce mistakes and keep the agent aligned with your expectations.
- Review and update instructions as your needs or available tools change.

Q: Why define scope?

A: It keeps the agent focused and clarifies permitted work.

Agent Roles and Responsibilities

- A role specifies the work an agent is responsible for.
- Each role has clear inputs, outputs and ownership.
- Different roles focus on different tasks, such as researching, writing and reviewing.
- Roles can exist in one agent (multi-role) or in several agents that work together.
- Using more agents can improve modularity and focus, but it also adds coordination overhead.

Common Agent Roles

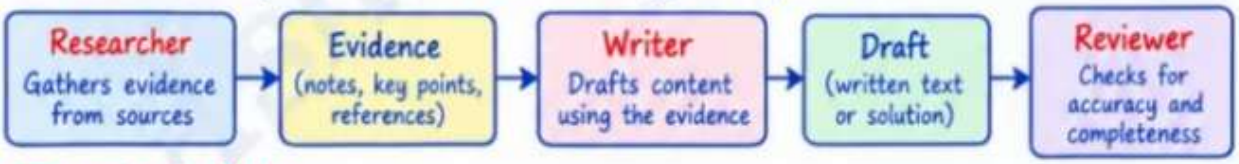
- **Researcher:** gathers and organises evidence from sources.
- **Writer:** uses the evidence to draft content or a solution.
- **Reviewer:** checks the draft for accuracy, completeness and consistency.

Inputs, Outputs and Ownership

- **Inputs:** information or data an agent receives from a user, tool or another agent
- **Outputs:** the result the agent produces, such as a list of notes or a written draft.
- **Ownership:** the agent is responsible for completing its task and producing the output in the expected format.

Example Workflow

- A Researcher agent gathers key points about a topic from given sources.
- A Writer agent turns the evidence into a clear draft.
- A Reviewer agent checks the draft and suggests improvements.



Important Note

- Choose the right number of agents for the task.
- More agents can improve separation of work and focus, but they require coordination such as passing outputs, managing order and handling errors.
- Clear roles and well-defined inputs and outputs help agents work together effectively.

Q: Why give agents clear roles?
A: Clear roles reduce duplicated work and support reliable handoffs.

Structured Outputs

- Structured outputs follow a specified format, such as JSON with defined fields.
- Using a structured format makes it easier to parse, validate and use the output in applications or downstream steps.
- Common format: JSON (JavaScript Object Notation) with named fields.

Example Output (JSON)

```
{
    "topic": "AI",
    "page": 18,
    "status": "draft"
}
```

Key Points

1. Define types and required fields

- Specify the expected fields, their types (e.g., string, number, boolean) and which fields are required.
- Helps ensure consistent and reliable outputs for downstream use.

2. Validate before use

- Check that the output follows the expected schema (correct field names and data types).
- Validation helps catch format errors early and prevents runtime issues.

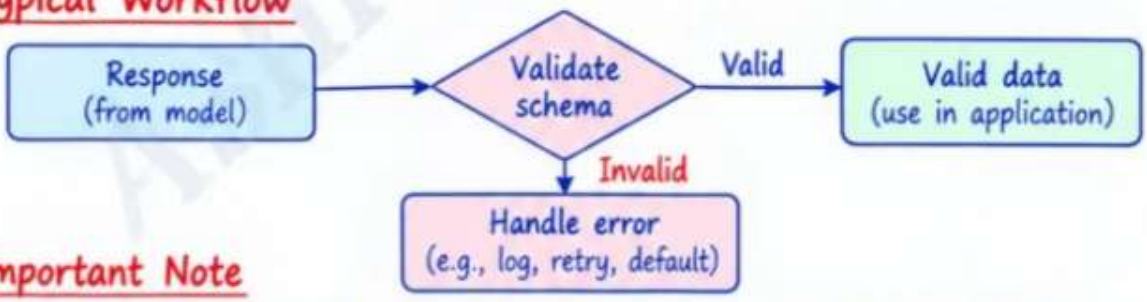
3. Handle invalid or missing values

- The model may sometimes return invalid types, missing fields or extra fields.
- Implement checks and handle such cases with defaults, error messages or retries.

4. Schema validity does not guarantee factual accuracy

- An output can be correctly formatted but still contain incorrect or misleading information.
- Always verify important information separately when needed.

Typical Workflow



Important Note

- Define a clear schema and validate the output before using it in your application.
- Even when the JSON is valid and matches the schema, the content may still be factually incorrect, so additional checks may be required.

Q: Does valid JSON guarantee accuracy?

A: No. Format correctness and factual correctness are separate.